



Cheese

A modern web-based chess analysis platform
powered by Stockfish.

AUTHOR	Vihaan Productions
VERSION	1.2
DATE	July 2026
LIVE	usecheese.xyz
LICENSE	GPL-3.0

TECHNICAL DOCUMENTATION • ARCHITECTURE, IMPLEMENTATION, OPERATIONS

Contents

Executive Summary · p.5

- The problem · p.5
- The approach · p.5
- What it does · p.6
- Target users · p.6
- Project vision · p.7
- Status at a glance · p.7

Product Overview · p.8

- The five surfaces · p.8
- Design principles · p.8
- Explicit non-goals · p.9
- Competitive position · p.9

Feature Reference · p.11

- 3.1 Home page · p.11
- 3.2 Analysis · p.11
- 3.3 Opening Explorer · p.13
- 3.4 Database · p.14
- 3.5 Training · p.14
- 3.6 Interface · p.14

System Architecture · p.16

- Architectural style · p.16
- Threading model · p.16
- Dependency inventory · p.17
- Separation of concerns · p.17

Folder Structure · p.19

- Root files · p.19
- Analysis/ · p.19
- Other page directories · p.20
- Directory naming · p.20

Chess Logic and the Move Tree · p.22

- Why a tree · p.22
- The GameNode structure · p.22
- Delegation of the rules · p.23
- Promotion · p.24
- Ending detection · p.24

Stockfish Integration · p.26

- Initialisation · p.26
- The analysis request · p.26
- Parsing the engine output · p.27
- The evaluation bar · p.28
- Configuration and its absence · p.29

Board Rendering and Interaction · p.30

- The rendering pipeline · p.30
- Piece assets · p.30
- Drag and drop · p.31
- Feedback: highlighting, sound, and check · p.31

Opening Explorer · p.32

- The data · p.32
- The rendering problem · p.32
- Chunking and the sentinel · p.33
- The reveal cascade · p.33
- Hand-off to Analysis · p.34
- Limitations and future work · p.34

Master Game Database · p.35

- The player registry · p.35
- Splitting a multi-game PGN · p.35
- Presentation · p.36
- Hand-off to Analysis · p.36
- Limitations · p.36

Training Mode · p.37

- The state machine · p.37
- Turn enforcement · p.37
- The engine as an opponent · p.37
- Game termination · p.38
- Board orientation · p.38
- The no-branching rule · p.38
- Future work · p.39

Persistence and the Data Model · p.40

- The three keys · p.40
- The one-shot hand-off pattern · p.40
- The saved-analysis library · p.41
- What is stored, and what is not · p.41
- Constraints of browser storage · p.42

User Interface and Design System · p.43

- Layout · p.43
- The design language · p.43
- Typography · p.44
- The sidebar · p.44
- The mobile gate · p.44

Game Flow · p.46

- Session flow · p.46
- The move flow · p.46
- Step by step · p.46
- The variation case · p.47

Performance · p.48

- The cost model · p.48
- What was actually optimised · p.48
- What was deliberately not optimised · p.49
- Memory · p.49
- Loading · p.49
- Future optimisation · p.49

Security and Privacy · p.50

- The threat model · p.50
- The one real risk: PGN as untrusted input · p.50
- Input validation · p.51
- Supply chain · p.51
- Privacy · p.51
- The data-loss risk · p.52

Accessibility · p.53

- Current state · p.53
- The remediation plan · p.53
- Why this matters for the roadmap · p.54

Deployment · p.55

- Hosting model · p.55
- Trap 1: Workers is not Pages · p.55
- Trap 2: the output directory must be the true root · p.55
- Trap 3: case sensitivity · p.56
- Trap 4: two different caches · p.56
- DNS · p.56
- The deployment checklist · p.57
- Recommended improvement: connect Git · p.57

Developer Guide · p.58

- Prerequisites · p.58
- Getting the code · p.58
- Running locally · p.58
- Common tasks · p.58
- Code conventions · p.59
- Testing · p.59

Internal API Reference · p.61

- 20.1 Core state and rendering · p.61
- 20.2 The move tree · p.62
- 20.3 Engine · p.63
- 20.4 Persistence · p.64
- 20.5 Known duplication · p.64

Technical Challenges and Lessons Learned · p.66

- Engine and state · p.66
- Interface · p.67
- Deployment · p.67
- What the project would do differently · p.68

Development Roadmap · p.69

Shipped · p.69
Version 1.3 — Puzzles and repayment · p.69
Version 2.0 — Mobile, and a stronger engine · p.69
Version 3.0 — Accounts and the cloud · p.70
Version 4.0 — Platform · p.70
Sequencing principle · p.71

Glossary · p.72

Chess terminology · p.72
Technical terminology · p.72

Appendix · p.74

A. Technology summary · p.74
B. Storage keys · p.74
C. UCI commands used · p.74
D. Resources and references · p.75
E. Licence · p.75
F. Credits and acknowledgements · p.75

About this document

This is the reference documentation for **Cheese**, a browser-based chess analysis and study platform. It describes the system as it exists at version 1.2 — what it does, how it is built, why it is built that way, and what remains to be done.

The document is written for several audiences at once. A developer joining the project should be able to read Parts II and V and become productive without reading the source first. A reviewer, employer, or contributor should be able to read Part I and understand the scope and the engineering decisions. And the author, returning to this code after a long gap, should be able to recover the full mental model from these pages alone.

Document conventions

Convention	Meaning
monospace	Identifiers, file paths, and literal code taken from the source tree.
Shipped	The feature exists in version 1.2 and is live at usecheese.xyz.
Planned	The feature is designed or intended but is <i>not</i> implemented in 1.2. It appears only in roadmap sections.
Callout boxes	Blue = context. Green = practical tip. Amber = a constraint or a trap that has already caused a bug once.

IMPORTANT

This document deliberately distinguishes what is **built** from what is **intended**. Documentation that describes aspirations as though they were features is worse than no documentation, because it destroys the reader's ability to trust any of it. Where version 1.2 falls short — mobile support, theming, accessibility — this document says so plainly.

Version history

Version	Status	Summary
1.0	Superseded	Analysis board, Stockfish evaluation, move list.
1.1	Superseded	Opening Explorer, PGN import, saved analyses.
1.2	Current	Training mode, master-game Database, Puzzles placeholder, mobile gate, public launch on usecheese.xyz.

CHAPTER 1

Executive Summary

What Cheese is, and why it exists

Cheese is a chess analysis and study platform that runs entirely inside the web browser. It pairs a full-strength Stockfish 18 engine with an interactive board, a branching analysis tree, an opening reference covering the complete ECO classification, and a curated database of master games — with no account, no server, and no installation.

The problem

Serious chess study tools tend to fall into two camps. Desktop engines and database programs are powerful but heavy: they must be installed, they are usually paid, and they carry two decades of interface debt. Web platforms are convenient but gated: the good analysis features sit behind accounts, subscriptions, and rate limits, and every position you look at is sent to someone else's server.

There is a third option that modern browsers make possible but that few tools actually take. WebAssembly can run Stockfish at meaningful strength on the client. `localStorage` can persist a study library. Static hosting can serve the whole thing for free. Nothing about a personal analysis session actually *requires* a backend — the requirement is an artifact of how these products are monetised, not of the problem itself.

The approach

Cheese is built on that observation. It is a pure client-side application: HTML, CSS, and vanilla JavaScript, with `chess.js` for rules and a Stockfish 18 Lite WebAssembly build for evaluation. There is no backend, no database, no build step, and no framework. The entire platform is a folder of static files served from a CDN edge.

This constraint is not a limitation that the project tolerates; it is the design decision the project is organised around. It produces a tool that is instant to open, free to run, private by construction, and impossible to break with a server outage — because there is no server.

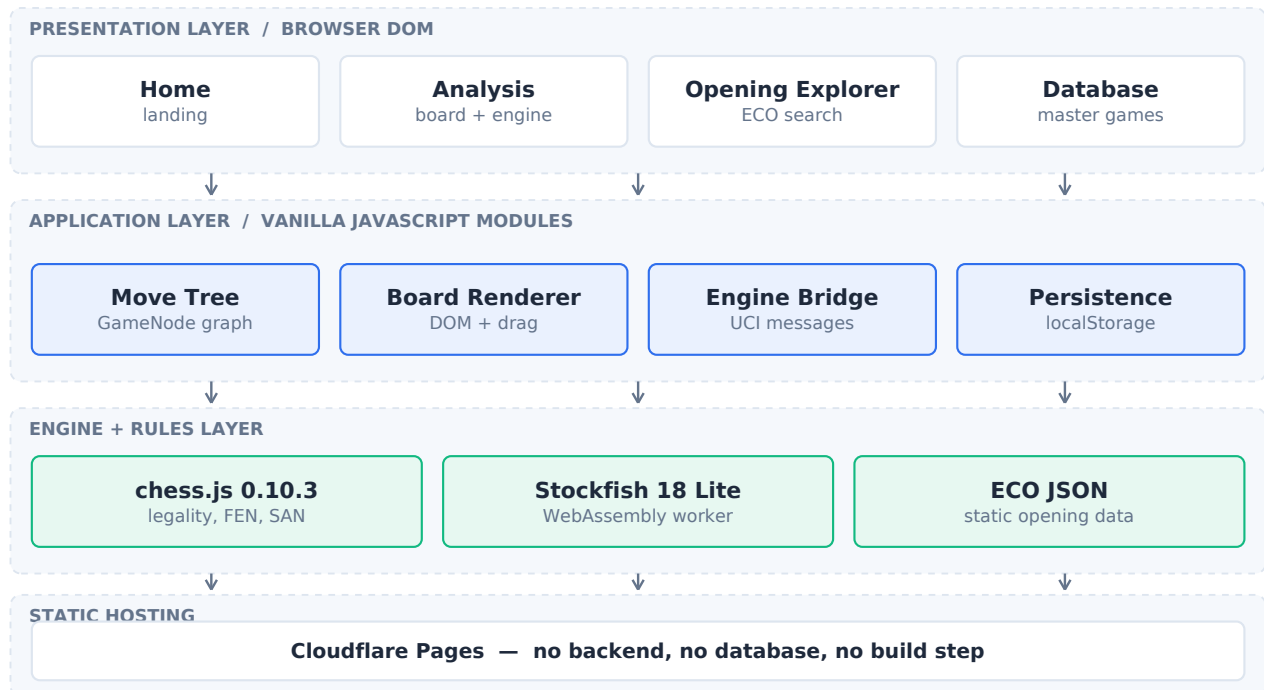


Figure 1.1 — System layers. Every layer above the hosting line executes in the user's browser.

What it does

Capability	Summary
Analysis board	Interactive board with drag-and-drop, legal-move highlighting, promotion, castling, en passant, and sound.
Engine evaluation	Stockfish 18 Lite runs in a Web Worker to depth 18, streaming evaluation and principal variation into the UI in real time.
Branching move tree	Analysis is stored as a tree, not a list. Any position can sprout a variation, and variations render inline.
Opening Explorer	The full ECO catalogue (volumes A-E) is searchable by name or code, and any opening loads directly into the board.
Master Database	Curated PGN collections for eight world-class players, parsed in the browser, one click from analysis.
Training mode	Full games against Stockfish with colour choice, resignation, and complete checkmate and draw detection.
Persistence	Analyses are saved to browser storage and reloaded on demand.

Target users

Cheese is aimed at the club and improving player — roughly 800 to 2000 rating — who wants engine analysis without ceremony. That user has a specific, recurring need: paste or replay a game, see where it went wrong, branch a few alternatives, and check what the opening was called. They do not need cloud sync, a social feed, or a subscription. They need a board and an engine, immediately.

Secondary audiences include coaches wanting a zero-friction board to demonstrate lines on, and developers who want a readable, framework-free reference for how a chess interface is actually assembled.

Project vision

The long-term goal is a complete self-contained study environment: analysis, openings, tactics, endgames, and game review, all running locally, all free, all open source. Version 1.2 delivers the analytical core of that vision. The roadmap in Chapter 21 sets out how the remaining pieces — puzzles, mobile support, richer game review — fit into it.

NOTE

Scope discipline. Cheese has deliberately *not* implemented user accounts, cloud storage, or online play. Each of those would require a backend, and a backend would compromise the property that makes the tool worth building. They are listed in the roadmap only under versions where a backend is explicitly on the table.

Status at a glance

Metric	Value
Version	1.2
Lines of application code	~7,500 (HTML, CSS, JavaScript)
Runtime dependencies	2 (chess.js, Stockfish WASM)
Build step	None
Backend services	None
Hosting	Cloudflare Pages (static)
Engine strength	Stockfish 18 Lite, depth 18 in analysis
Licence	GPL-3.0

CHAPTER 2

Product Overview

Users, scope, and the shape of the product

The five surfaces

Cheese is organised as five pages, each of which is a self-contained static page with its own stylesheet and script. They share a sidebar, a visual language, and a small number of `localStorage` keys, but they share no runtime state and no build pipeline. A page can be modified, broken, or replaced without affecting the others.

Surface	Route	Purpose
Home	/	Landing page. Explains the product and routes the user into a tool.
Analysis	/Analysis/	The core workspace: board, engine, evaluation, move tree, saved games.
Opening Explorer	/opening-explorer/ /	Search the ECO catalogue; hand an opening off to Analysis.
Training	/Training/	Play a complete game against Stockfish.
Database	/Database/	Browse master games by player; hand a game off to Analysis.
Puzzles	/Puzzles/	Placeholder in 1.2. Reserved route for the tactics trainer.



Figure 2.1 — User journey. Analysis is the hub; every other surface eventually routes into it.

Design principles

Four principles govern the product, and they are worth stating because they explain most of the decisions documented later.

1. The board is the product

Everything that is not the board or the evaluation is chrome, and chrome should recede. The interface is dark, low-contrast in its furniture, and high-contrast only where information lives. There are no banners, no notifications, and no upsells.

2. No round trips

Any feature that would require asking a server for permission or data is either redesigned to run locally or deferred. This is why the ECO database is a set of static JSON files rather than an API call, and why saved analyses live in `localStorage` rather than in a user account.

3. Honest engine output

The evaluation shown is the engine's, unfiltered and unsmoothed. Cheese does not round scores into vague verdicts or hide the depth. The user sees the number, the depth it was reached at, and the line that produced it.

4. Analysis is a tree, not a list

Real analysis branches. A tool that models a game as a flat list of moves forces the user to destroy one line in order to explore another. Cheese models the game as a tree from the ground up — this single decision shapes the entire data model and is the subject of Chapter 6.

Explicit non-goals

Stating what a product refuses to do is as useful as stating what it does. As of 1.2, Cheese does not attempt any of the following, and features are rejected if they require them:

- **User accounts.** No sign-up, no identity, no server-side profile.
- **Online multiplayer.** No matchmaking, no real-time opponent, no game server.
- **Cloud storage.** Analyses live in the browser that created them.
- **Telemetry.** No analytics, no tracking, no third-party scripts beyond the chess.js CDN reference.
- **Monetisation.** No advertising and no paywalled features.

NOTE

Two of these — accounts and cloud storage — appear in the version 3.0 roadmap. That is not a contradiction: the roadmap explicitly marks 3.0 as the release where the no-backend constraint is *consciously traded away*, in exchange for cross-device sync. Until then, the constraint holds.

Competitive position

	Cheese	Desktop GUI	Major web platforms
Installation	None	Required	None
Cost	Free	Often paid	Freemium
Engine	Local WASM	Local native	Server-side, rationed
Privacy	Positions never leave the device	Local	Sent to server
Analysis depth	User-controlled	User-controlled	Often capped
Offline	Yes, after first load	Yes	No

	Cheese	Desktop GUI	Major web platforms
Strength ceiling	Lower (WASM overhead)	Highest	High

The honest trade is in the last row. A native Stockfish binary with multiple threads and a large hash table will out-analyse a single-threaded WASM build every time. Cheese accepts a lower strength ceiling in exchange for zero friction, and for the club player that is the correct trade — depth 18 is far beyond what is needed to find the mistakes in a 1500-rated game.

CHAPTER 3

Feature Reference

Every user-facing capability, with implementation notes

This chapter enumerates each feature: what it is for, how it is built, what it buys the user, and where it could go next. Features not present in 1.2 are marked accordingly and are described in the future tense.

3.1 Home page

Home

SHIPPED

Purpose	Orient a first-time visitor in one screen and route them into the right tool.
Implementation	A single static page (<code>index.html</code> at the deployment root) with a hero block, two rows of call-to-action buttons, and the shared sidebar. No JavaScript logic beyond navigation.
Benefits	Gives the project a front door that explains itself, which matters disproportionately for an open-source tool being evaluated by a stranger.
Future work	Add a live board preview or a short animated demonstration of the analysis loop.

3.2 Analysis

Analysis board

SHIPPED

Purpose	Provide an interactive chessboard that enforces the rules and records everything played on it.
Implementation	Sixty-four <code><div class="square"></code> elements. Pieces are absolutely-positioned PNGs re-placed from the current FEN on every repaint. Legality is delegated entirely to <code>chess.js</code> ; the renderer never makes a rules decision. Custom pointer-based drag-and-drop with snap-back animation on illegal drops.
Benefits	No board library dependency, complete control over animation and highlighting, and a rendering path simple enough to debug by reading it.
Future work	Extract the board into a reusable module so Training and a future Puzzles page stop duplicating its logic.

Stockfish integration

SHIPPED

Purpose	Evaluate the current position continuously and surface the engine's best line.
Implementation	Stockfish 18 Lite compiled to WebAssembly, instantiated as a dedicated Worker. Communication is raw UCI over <code>postMessage</code> . Analysis requests are debounced by 80 ms and always preceded by an explicit stop. See Chapter 7.

Benefits	Full-strength analysis with zero server cost and complete privacy; the UI never blocks because the search runs off the main thread.
Future work	Expose depth and MultiPV as user settings; add an explicit engine on/off toggle to save battery.

Evaluation bar

SHIPPED

Purpose	Communicate the engine's assessment at a glance, without the user parsing a number.
Implementation	The raw centipawn score is parsed from the UCI info line, normalised to White's perspective (Stockfish reports relative to the side to move), divided by 100, and mapped to a bar height. Mate scores bypass the mapping and pin the bar to an extreme. See Figure 7.2.
Benefits	Turns a raw engine number into a spatial signal the eye reads instantly.
Future work	Add a smoothed evaluation graph across the whole game, which is the single most requested feature in comparable tools.

Move list and branching tree

SHIPPED

Purpose	Record the game, allow navigation to any earlier position, and permit exploring alternatives without discarding the original line.
Implementation	A GameNode tree. Each node holds a move, the resulting FEN, a parent pointer, and a list of children. The first child is the mainline; subsequent children render as inline variations. Fully recursive rendering. See Chapter 6.
Benefits	This is the feature that separates Cheese from a toy board. Analysis is inherently exploratory, and a tree is the honest data structure for it.
Future work	Add variation promotion (make a sideline the mainline), collapsing, and comment annotation per node.

Move navigation

SHIPPED

Purpose	Step through a game move by move, or jump directly to any position.
Implementation	Previous / next / first / last controls plus direct click on any node in the rendered tree. Navigation sets currentNode and calls refreshUI(); the board is rebuilt from that node's stored FEN, so navigation cannot desynchronise from the game state.
Benefits	Cheap, correct, and impossible to get out of sync — the FEN is the single source of truth for what is drawn.
Future work	Keyboard arrow-key binding, which is a small change with a large usability return (see Chapter 17).

PGN import

SHIPPED

Purpose	Load an external game for review.
Implementation	PGN text is handed to chess.js for parsing, then replayed move by move into a fresh GameNode tree so that every intermediate FEN is materialised. Games arriving from the Database page are passed via the cheeseImportPGN storage key.
Benefits	The single most common real workflow — 'here is my game, tell me what went wrong' — works end to end.
Future work	Accept drag-and-drop of .pgn files and support multi-game PGN files with a game picker.

PGN export**SHIPPED**

Purpose	Get a game out of Cheese and into any other chess tool.
Implementation	generatePGNFromNode() walks from the given node back to the root, collecting moves, then emits standard SAN with move numbers. Exposed through a copy-to-clipboard control.
Benefits	Prevents lock-in. Anything studied in Cheese can leave in a universal format.
Future work	Include variations and comments in the exported PGN (currently the mainline to the current node is exported), and offer file download in addition to clipboard.

Save system**SHIPPED**

Purpose	Persist analyses across sessions without an account.
Implementation	Analyses are serialised and stored in localStorage under the cheeseSavedAnalyses key, listed in a Games tab beside the analysis panel, and restored on click. See Chapter 12.
Benefits	Gives the tool memory — the difference between a scratchpad and a study library — with no infrastructure.
Future work	Export the whole library to a file, and warn the user that browser storage can be cleared (see Chapter 16).

3.3 Opening Explorer

Opening Explorer**SHIPPED**

Purpose	Let a user find an opening by name or ECO code and load it onto the board.
Implementation	The five ECO volume files (ECO/ecoA.json ... ecoE.json) are fetched once on page load and held in memory. Search filters the array; results render in chunks of 60 cards, with an IntersectionObserver sentinel pulling the next chunk into the DOM as the user scrolls. See Chapter 9.

Benefits	The complete ECO catalogue is browsable at 60fps with no server, because the DOM never holds more than what has been scrolled to.
Future work	Add move-tree navigation of the opening (walk the line move by move) and, eventually, real master-game frequency statistics per position.

3.4 Database

Master game database

SHIPPED

Purpose	Provide a curated library of instructive games from world-class players.
Implementation	A data-driven PLAYERS registry maps each player to a portrait and a PGN file. Multi-game PGN files are split on blank-line boundaries and their headers parsed with regular expressions to build the game list. Clicking a game writes it to <code>cheeseImportPGN</code> and navigates to Analysis. See Chapter 10.
Benefits	Adding a player is a data change, not a code change — drop in a portrait and a PGN, add one registry entry.
Future work	Add search and filtering across all games (by opening, result, or year), and grow the collection substantially.

3.5 Training

Play against Stockfish

SHIPPED

Purpose	Let the user play a complete, properly terminated game against the engine.
Implementation	A second, dedicated Stockfish worker, separate from the analysis engine. Turn ownership is enforced in both the click and drag handlers. The engine replies with <code>go movetime 700</code> . Game termination is detected by replaying the mainline into a fresh <code>chess.js</code> instance and testing every ending condition. See Chapter 11.
Benefits	Closes the study loop: play a game, then analyse it, without leaving the site.
Future work	Add engine strength levels (currently only full strength), a clock, and an automatic hand-off of the finished game into Analysis for review.

3.6 Interface

Modern interface

SHIPPED

Purpose	Present a calm, consistent, dark glassmorphism environment across every page.
Implementation	Hand-written CSS with a shared visual language: translucent panels, blurred backdrops, a restrained accent palette, and one background illustration used site-wide. No CSS framework. See Chapter 13.

Benefits	A cohesive product feel that makes a hobby project read as a considered one.
Future work	Extract the repeated sidebar and panel rules into a shared stylesheet; they are currently duplicated per page (see Chapter 20).

Board and piece themes

PLANNED — NOT IN 1.2

Purpose	Allow the user to choose board colours and piece sets.
Implementation	Not implemented in 1.2. The board currently uses a single brown SVG square pair and one piece set. The rendering path already routes every piece image through <code>getPieceImage()</code> , which is the natural seam for theming.
Benefits	—
Future work	Introduce a theme object mapping piece codes to asset paths and board colours to CSS custom properties; persist the selection in <code>localStorage</code> .

Responsive and mobile support

PLANNED — NOT IN 1.2

Purpose	Make the platform usable on phones and tablets.
Implementation	Not implemented in 1.2. The layout assumes a desktop viewport. A shared <code>mobile-gate.css</code> currently detects viewports under 768 px and replaces the page with a courteous 'coming soon' notice rather than presenting a broken interface.
Benefits	—
Future work	A genuine responsive rebuild: fluid board sizing, a collapsible sidebar, a bottom-sheet analysis panel, and touch-native piece movement. This is the highest-priority item on the roadmap.

IMPORTANT

The mobile gate is a *deliberate* non-feature. Shipping a desktop layout to a phone produces an unusable board and a bad first impression that the user never returns from. Blocking the experience honestly, and saying why, is the better failure mode until the responsive work is actually done.

CHAPTER 4

System Architecture

How the pieces fit, and what runs where

Cheese has an unusually simple architecture, and that simplicity is load-bearing. Understanding it takes about ten minutes, which is the point.

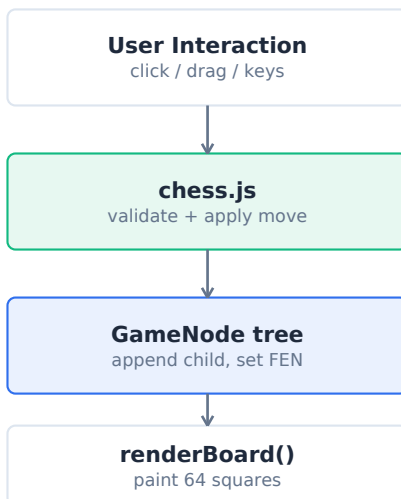
Architectural style

The system is a **static multi-page application**. Each page is an independent HTML document with its own script and stylesheet. There is no client-side router, no shared bundle, no global application object, and no build step. Navigation between pages is an ordinary browser navigation, and each page boots from scratch.

This is an old-fashioned architecture, chosen deliberately over a single-page application. The benefits are concrete: a bug in Training cannot crash Analysis; any page can be opened directly by URL; the browser's back button works correctly with no effort; and the entire deployment is a folder of files that can be copied onto any static host.

The cost is duplication — the sidebar markup and several CSS rules are repeated across pages. Chapter 20 discusses this honestly as accumulated debt, and Chapter 21 proposes the shared-stylesheet extraction that pays it down.

MAIN THREAD



DEDICATED WORKER THREAD

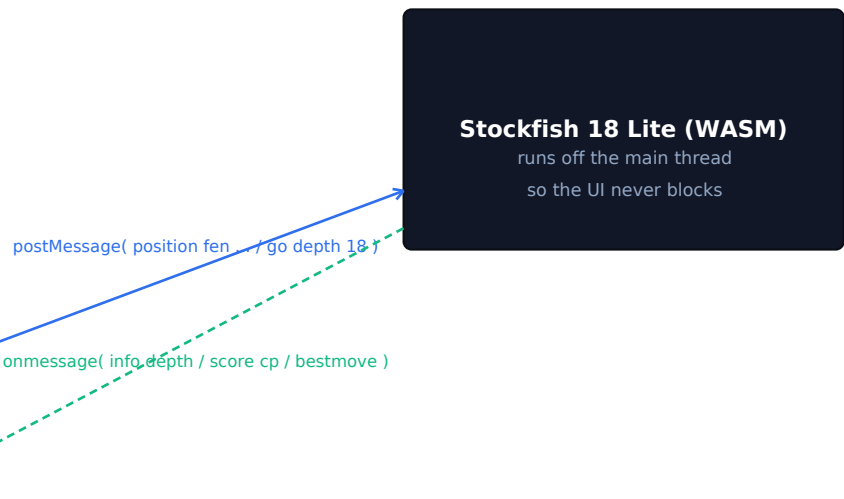


Figure 4.1 — Runtime threading model. The engine's search never touches the thread that paints the board.

Threading model

This is the one genuinely non-trivial piece of the architecture. A chess engine search is a long, CPU-bound computation. Run on the main thread, it would freeze the interface — pieces would not drag, clicks would queue, and the page would be reported as unresponsive.

Stockfish therefore runs inside a dedicated Worker, which the browser schedules on a separate thread. The main thread and the worker share no memory and communicate only by passing messages. The main thread sends UCI commands; the worker streams UCI output back.

Everything the user can see or touch stays on the main thread and stays responsive, no matter how hard the engine is working.

JAVASCRIPT

```
// Analysis/script.js - the entire engine boot sequence
const engine = new Worker("stockfish/stockfish.js");
engine.postMessage("uci");
engine.postMessage("isready");

engine.onmessage = function (event) {
  const line = event.data;          // raw UCI text
  if (line.includes("score cp")) { /* update eval bar */ }
  if (line.includes("score mate")) { /* forced mate found */ }
  if (line.includes("bestmove")) { /* search complete */ }
};
```

IMPORTANT

Training instantiates its *own* engine worker rather than sharing the analysis one. This was not premature separation — it fixed a real bug. A single shared engine had its search interrupted by the analysis debounce timer mid-game, causing the bot to occasionally never reply. Two workers, two independent search lifecycles, no interference. See Chapter 21.

Dependency inventory

The project has exactly two runtime dependencies. Both are chosen so that they can be understood, replaced, or removed.

Dependency	Version	Role	Delivery
chess.js	0.10.3	Move generation, legality, check and mate detection, FEN and SAN handling.	CDN script tag
Stockfish	18 Lite	Position evaluation and best-move search.	Local WASM + JS glue, run as a Worker

Notably absent is a board rendering library. Cheese does **not** use chessboard.js or any equivalent: the board is rendered directly to the DOM by roughly 100 lines of application code. This was a deliberate choice — board libraries impose their own animation and interaction models, and the custom highlighting, glide animation, and promotion flow that Cheese wanted were easier to write from scratch than to fight a library into producing.

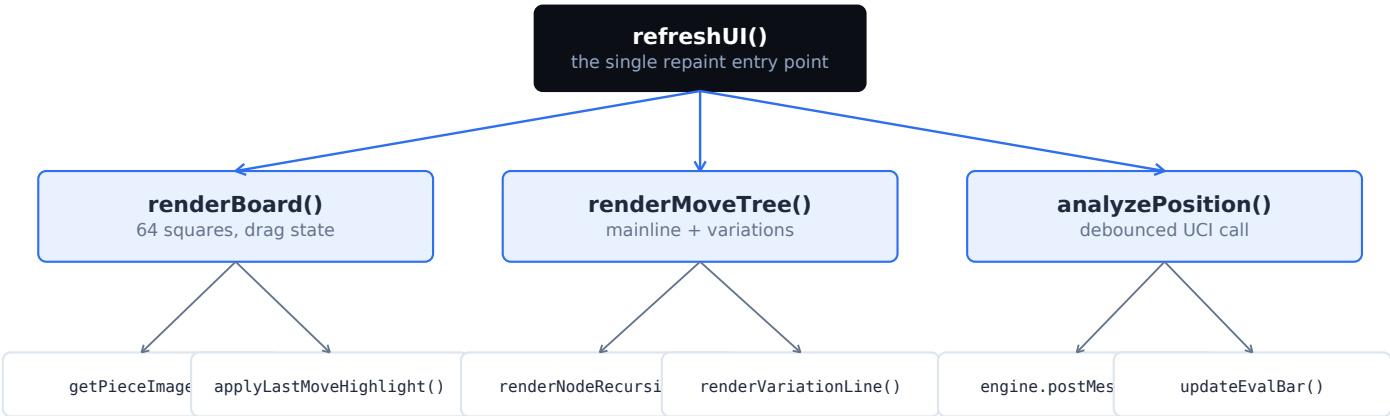
Separation of concerns

The cleanest way to understand the codebase is by the boundary each module refuses to cross.

Concern	Owner	Never does
Chess rules	chess.js	Never touches the DOM or knows the board exists.
Evaluation	Stockfish worker	Never mutates application state; only emits text.
Game history	GameNode tree	Never renders; holds no DOM references.

Concern	Owner	Never does
Rendering	<code>renderBoard()</code> , <code>renderMoveTree()</code>	Never decides legality; asks <code>chess.js</code> .
Persistence	<code>localStorage</code> helpers	Never triggers a repaint directly.

The important invariant: **the renderer is a pure function of the current node's FEN**. It never accumulates state, never applies a move itself, and never has an opinion about whether a move was legal. If the board on screen is wrong, the bug is upstream in the tree, not in the paint code — which makes an entire class of desynchronisation bug structurally impossible.



one function repaints everything — no framework, no virtual DOM, no reconciliation

Figure 4.2 — Component relationships beneath the single repaint entry point.

CHAPTER 5

Folder Structure

The deployed tree, file by file

The deployed tree *is* the source tree. There is no compilation, so what you read in the repository is byte-for-byte what the browser executes — which makes debugging in production the same activity as debugging locally.

<code>usecheese.xyz/</code>	deployment root (Cloudflare Pages)
└ <code>index.html</code>	Home page — served at /
└ <code>style.css</code>	Home styling
└ <code>mobile-gate.css</code>	shared small-viewport gate
└ <code>Background/</code>	Home background art
└ <code>Analysis/</code>	the analysis workspace
└ <code>index.html</code>	board, panel, eval bar markup
└ <code>script.js</code>	1,812 lines — tree, board, engine bridge
└ <code>style.css</code>	1,860 lines — board + panel styling
└ <code>pieces/</code>	12 piece PNGs
└ <code>board/</code>	light / dark square SVGs
└ <code>sounds/</code>	move, capture, castle, check, promote
└ <code>stockfish/</code>	stockfish.js + .wasm
└ <code>Training/</code>	play against Stockfish
└ <code>script.js</code>	2,068 lines — game loop, bots, game-over
└ <code>opening-explorer/</code>	ECO opening search
└ <code>ECO/</code>	ecoA.json ... ecoE.json
└ <code>Database/</code>	master game collections
└ <code>Magnus/, Fisher/, Tal/, ...</code>	portrait + .pgn per player
└ <code>Puzzles/</code>	placeholder page (v1.3 target)

Figure 5.1 — Deployment tree. Blue entries are directories.

Root files

Path	Purpose
<code>index.html</code>	The Home page. Because it sits at the deployment root, it is what usecheese.xyz/ serves.
<code>style.css</code>	Home page styling only.
<code>mobile-gate.css</code>	Shared across every page. Hidden by default; below 768 px it hides the application and shows the 'coming soon' card.
<code>Background/</code>	The Home background illustration.
<code>README.md</code>	Repository front page.
<code>LICENSE</code>	GPL-3.0 text.

Analysis/

The largest and most important directory. It is fully self-contained: it owns its own copies of the piece art, board squares, sounds, and engine binary, and depends on nothing outside itself except the shared mobile gate.

Path	Lines	Purpose
index.html	470	Board markup (64 squares), evaluation bar, analysis panel, sidebar, promotion picker, modals.
script.js	1,812	The application core: GameNode tree, board rendering, drag-and-drop, engine bridge, PGN generation, save system.
style.css	1,860	Board, panel, eval bar, move tree, modal, and animation styling.
pieces/	—	Twelve piece PNGs, named {w b}_{type}_png_shadow_512px.png.
board/	—	Light and dark square SVGs.
sounds/	—	move-self, capture, castle, move-check, promote.
stockfish/	—	stockfish.js glue plus the .wasm binary.

Other page directories

Directory	Contents
Training/	index.html, script.js (2,068 lines — the largest single file), style.css, its own stockfish/ and Bots/ avatars.
opening-explorer/	Page files plus ECO/, holding ecoA.json through ecoE.json.
Database/	Page files plus one directory per player, each holding a portrait image and a multi-game .pgn file.
Puzzles/	Placeholder page. script.js is an empty scaffold awaiting the 1.3 tactics engine.

IMPORTANT

Case sensitivity is not cosmetic. Local development on Windows or macOS uses a case-insensitive filesystem; Cloudflare's servers are case-sensitive Linux. A link to pieces/ that resolves happily against a folder named Pieces/ on your laptop will return a 404 in production. This cost real debugging time during the 1.2 launch and is documented as a lesson in Chapter 21.

Directory naming

The tree carries a visible inconsistency: some directories are capitalised (Analysis/, Training/, Database/) while one is lowercase-hyphenated (opening-explorer/). This is a scar, not a scheme. The directory was originally named Opening Explorer/ — with a space — which is legal on a local filesystem but fragile in URLs. It was renamed during deployment, and the lowercase form was adopted for the new name because it is the correct convention for web paths.

The right end state is that every directory follows the lowercase-hyphenated convention. That rename is deferred rather than done, because renaming a directory requires updating every link

that points at it, and doing so casually is exactly how the 404s of Chapter 21 were caused in the first place.

CHAPTER 6

Chess Logic and the Move Tree

The data model that makes analysis possible

This chapter covers the single most consequential design decision in Cheese: representing a game as a tree rather than a list.

Why a tree

The naive representation of a chess game is an array of moves. It is simple, it matches how a game is written down, and it is wrong for an analysis tool.

Consider the actual workflow. A user replays their game, reaches move 14, and wonders what would have happened if they had played a different bishop move. With an array, exploring that alternative means truncating the array — destroying moves 14 through 40 — and the original game is gone. The user must choose between remembering what they played and exploring what they should have played.

A tree removes the choice. The alternative move becomes a *second child* of the same parent position. Both lines exist simultaneously; the user can walk into either and back out again. This is how every serious chess program models analysis, and it is the reason Cheese can call itself an analysis tool rather than a board.

MAINLINE



VARIATION (sibling child of the same parent)



every node stores: move (SAN) • fen • parent • children[] • engineEval • engineLine • ply

Figure 6.1 — The move tree. The first child of a node is the mainline; additional children are variations.

The GameNode structure

JAVASCRIPT

```

1  class GameNode {
2    constructor({ move = null, fen, parent = null }) {
3      this.move      = move;      // chess.js move object (SAN, from, to, flags)
4      this.fen       = fen;       // position AFTER this move - the source of truth
5      this.parent    = parent;    // null only for the root
6      this.children  = [];        // children[0] is the mainline continuation
7      this.engineEval = null;     // cached evaluation, if the engine has reached it
8      this.engineLine = [];       // cached principal variation
9      this.comments  = "";        // reserved for annotation
10     this.ply       = parent ? parent.ply + 1 : 0;
11   }
12
13   addChild(node)    { this.children.push(node); return node; }
14   findChildBySAN(san) { return this.children.find(c => c.move.san === san); }
15   getPath()         { /* walk parents to root, return root-first array */ }
16 }

```

Three properties of this structure deserve comment.

The FEN is stored, not recomputed

Every node caches the complete position that results from its move. This costs a little memory and buys an enormous amount of correctness: navigating to any node is a matter of reading `node.fen` and painting it. There is no replay, no incremental application of moves, and therefore no possibility of the board drifting out of sync with the game after a long navigation session. Desynchronisation bugs — the classic plague of chess interfaces — are eliminated structurally.

children[0] is the mainline, by convention

There is no `isMainline` flag. The first child is the mainline because it was played first. Variations are simply the second and subsequent children. This keeps the structure minimal, at the cost of making 'promote this variation to the mainline' a reordering operation rather than a flag flip — an acceptable trade, and the reason promotion is a roadmap item rather than a shipped one.

The parent pointer makes PGN generation trivial

Because every node knows its parent, producing the PGN for any position is a walk up the tree, reversed. No separate history array is maintained, so no separate history array can fall out of sync.

JAVASCRIPT

```

function generatePGNFromNode(node) {
  const path = [];
  let cur = node;
  while (cur && cur.move) { path.unshift(cur.move.san); cur = cur.parent; }
  // path now holds the mainline SAN sequence from the root to `node`
  return formatWithMoveNumbers(path); // "1. e4 e5 2. Nf3 Nc6 ..."
}

```

Delegation of the rules

Cheese implements **no chess rules of its own**. Every question of legality is answered by `chess.js`, and the answers are treated as final. This is worth stating explicitly because the temptation to hand-roll 'just the simple cases' is strong and always ends badly — the simple cases are where en passant lives.

Rule	Handled by	How Cheese uses it
Legal move generation	<code>chess.js moves()</code>	Populates the legal-move dots shown on piece selection.
Move application	<code>chess.js move()</code>	Returns a move object or null; null means the drag snaps back.
Check	<code>chess.js in_check()</code>	Triggers the king-square flash animation.
Checkmate	<code>chess.js in_checkmate()</code>	Ends the game; the winner is the side that did <i>not</i> just move.
Stalemate	<code>chess.js in_stalemate()</code>	Draw.
Threefold repetition	<code>chess.js in_threefold_repetition()</code>	Draw.
Fifty-move rule	<code>chess.js in_draw()</code>	Draw.
Insufficient material	<code>chess.js insufficient_material()</code>	Draw.
Castling, en passant	<code>chess.js move flags</code>	Read from the returned move object to pick the correct sound.

Promotion

Promotion is the one place where a move cannot be completed from board geometry alone — the engine needs to know *what* the pawn becomes. Cheese detects the case before applying the move, suspends the interaction, and presents a picker.

JAVASCRIPT

```
function isPromotionMove(from, to) {
  const piece = chess.get(from);
  if (!piece || piece.type !== "p") return false;
  const rank = to[1];
  return (piece.color === "w" && rank === "8") ||
    (piece.color === "b" && rank === "1");
}

// on drop: if promotion, stash the pending move and open the picker
// instead of calling playMove() immediately
if (isPromotionMove(from, to)) {
  pendingPromotion = { from, to };
  showPromotionPicker(from, to);
  return; // playMove() is called by the picker's click handler
}
```

Ending detection

Game termination is checked by replaying the mainline into a *fresh* `chess.js` instance rather than by interrogating the live one. This looks wasteful and is intentional: the live instance is mutated during navigation and variation exploration, so its internal history — which is what `in_threefold_repetition()` and the fifty-move counter depend on — cannot be trusted to reflect the game actually played. A clean replay produces a clean verdict.

TIP

This is a real bug that was fixed, not a hypothetical. Threefold repetition was being reported incorrectly because the shared chess.js instance had absorbed moves from abandoned variations. Replaying into a fresh instance made the detection correct in one change.

CHAPTER 7

Stockfish Integration

Speaking UCI to a WebAssembly engine

Stockfish is not a library with an API. It is a program that reads commands on standard input and writes results to standard output, speaking a plain-text protocol called UCI. Integrating it in a browser means reproducing that conversation over `postMessage`.

Initialisation

JAVASCRIPT

```
const STOCKFISH_VERSION = "Stockfish 18 Lite";
const engine = new Worker("stockfish/stockfish.js");

engine.postMessage("uci"); // announce protocol; engine replies with its options
engine.postMessage("isready"); // engine replies "readyok" once initialised
```

The worker is created once, at page load, and lives for the lifetime of the page. Instantiating a WebAssembly engine is expensive — hundreds of milliseconds — so it must never be done per-analysis.

The analysis request

Every position change triggers an analysis request. The implementation is eleven lines and every one of them is doing something.

JAVASCRIPT

```
1 let analysisTimeout = null;
2
3 function analyzePosition() {
4   clearTimeout(analysisTimeout);
5
6   // Stop immediately so Stockfish is never mid-search when the new command arrives
7   engine.postMessage("stop");
8
9   analysisTimeout = setTimeout(() => {
10     engine.postMessage("position fen " + currentNode.fen);
11     engine.postMessage("go depth 18");
12   }, 80);
13 }
```

The stop is not optional

UCI engines are stateful. Sending position while a search is running produces undefined behaviour — in practice, the engine may continue reporting results for the *previous* position, which surfaces as an evaluation bar that shows a score belonging to a position the user has already navigated away from. The explicit `stop`, sent synchronously and outside the timer, guarantees the engine is idle before the new position arrives.

The debounce is not optional either

A user clicking rapidly through a game generates a position change every few dozen milliseconds. Without the 80 ms debounce, each one would start and immediately abort a search, and the engine would spend its entire life in start-up cost and never actually search anything. The debounce means that only the position the user *settles* on gets analysed. Eighty milliseconds is below the threshold of perception, so the analysis still feels instantaneous.

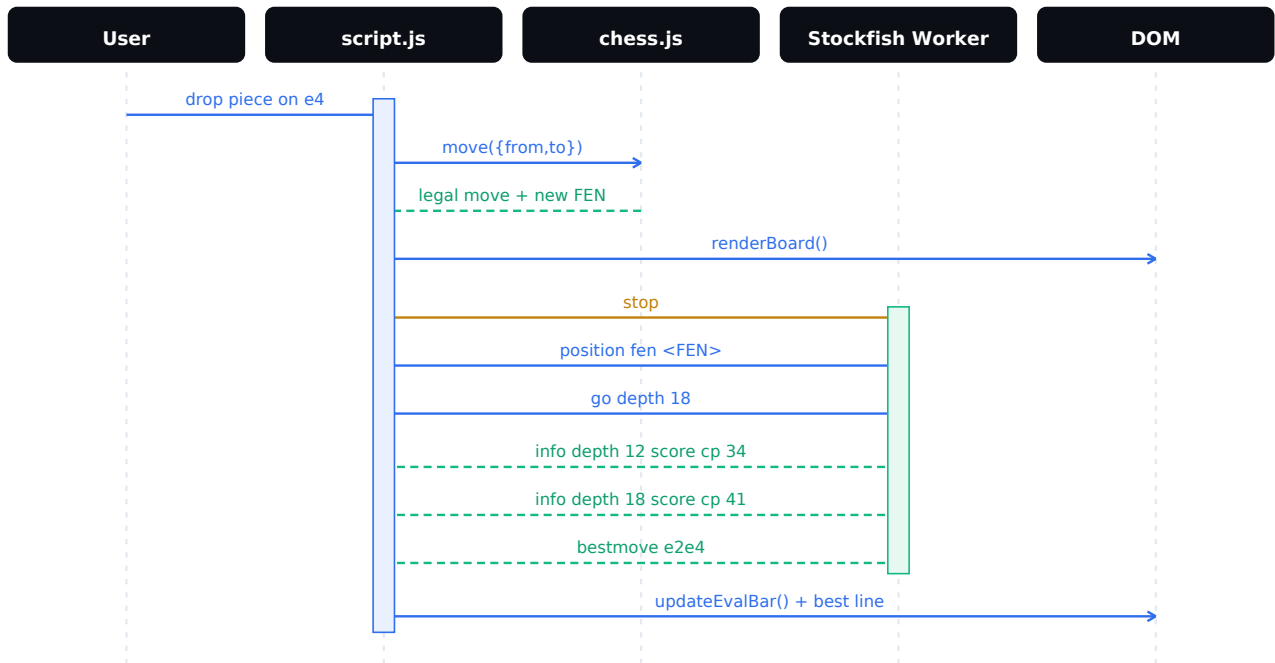


Figure 7.1 — Sequence of a single move, from drop to evaluation.

Parsing the engine output

Stockfish streams `info` lines continuously as its search deepens, then a final `bestmove`. Cheese parses three things out of the stream: depth, score, and the principal variation.

JAVASCRIPT

```

1 engine.onmessage = function (event) {
2   const line = event.data;
3
4   if (line.includes("depth")) {
5     const m = line.match(/depth (\d+)/);
6     if (m) engineDepth.textContent = "depth=" + m[1] + " | " + STOCKFISH_VERSION;
7   }
8
9   if (line.includes("score cp")) {
10    const m = line.match(/score cp (-?\d+)/);
11    if (m) {
12      let score = parseInt(m[1]);
13
14      // Stockfish reports from the side to move; convert to white-relative
15      const tmp = new Chess(currentNode.fen);
16      if (tmp.turn() === "b") score = -score;
17
18      score = (score / 100).toFixed(1);
19      currentNode.engineEval = score;
20      evalScore.textContent = parseFloat(score) > 0 ? "+" + score : score;
21      updateEvalBar(score);
22    }
23  }
24
25  if (line.includes("score mate")) { /* forced mate - see below */ }
26 };

```

IMPORTANT

The sign convention trap. UCI scores are reported from the perspective of the side to move, not from White's. A score of +150 with Black to move means *Black* is winning by 1.5 pawns. Every chess interface gets this wrong at least once, and the symptom is an evaluation bar that flips direction on every move. The fix is the three-line normalisation above: construct a throwaway chess.js instance from the FEN, ask whose turn it is, and negate if Black.

The evaluation bar

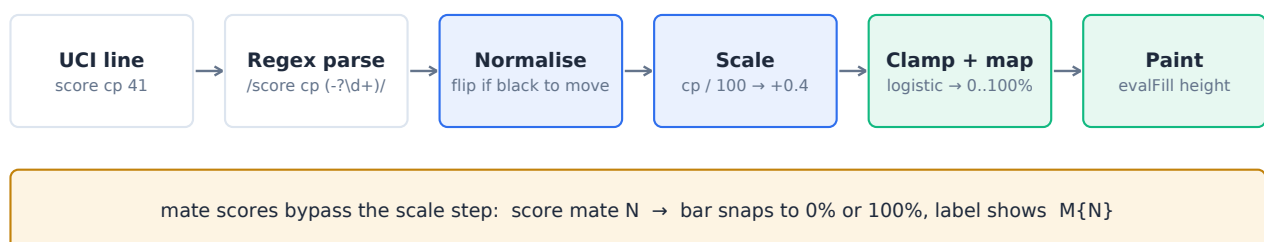


Figure 7.2 — From UCI text to a painted bar.

The mapping from centipawns to bar height is not linear, and it should not be. The difference between +0.2 and +0.5 is meaningful to a player; the difference between +8.0 and +9.0 is not — both mean 'completely winning'. A linear bar would waste most of its travel on distinctions nobody cares about. The bar therefore uses a compressive mapping that gives generous travel near equality and saturates at large advantages, and it clamps rather than overflowing.

Mate scores are handled as a separate case entirely. score mate 3 is not a centipawn value and must not be pushed through the numeric mapping; the bar pins to the extreme and the label

reads M3.

Configuration and its absence

UCI option	Cheese setting	Rationale
go depth	18 (analysis)	Deep enough to be trustworthy for club-level play; shallow enough to arrive quickly on a laptop.
go movetime	700 ms (training)	Fixed thinking time gives the bot a natural, consistent response rhythm.
MultiPV	Not set (default 1)	Only the top line is displayed, so requesting alternatives would cost search time for nothing.
Threads	Not set	The Lite WASM build is single-threaded; multi-threading requires SharedArrayBuffer and specific cross-origin isolation headers.
Hash	Not set (default)	Left at the engine default; tuning is a roadmap item.

The Threads row is the most consequential limitation in the system. A multi-threaded Stockfish would be several times stronger at the same wall-clock time, but enabling it requires serving the site with `Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy` headers so that `SharedArrayBuffer` becomes available. Cloudflare Pages can set those headers via a `_headers` file, so this is genuinely achievable — it is the highest-value engine improvement available and is scheduled for 2.0.

CHAPTER 8

Board Rendering and Interaction

Sixty-four divs and a drag handler

The board is the part of the system the user actually touches, and it is written from scratch. This chapter explains how it paints and how it responds.

The rendering pipeline



the previous piece positions are captured first, so each piece can be animated from its old square to its new one

Figure 8.1 — One pass of `renderBoard()`.

Rendering is a full repaint: the board is cleared and rebuilt from the current FEN on every change. There is no diffing and no virtual DOM. For a 64-square board with at most 32 pieces this is comfortably fast — the entire pass is well under a frame — and it buys the guarantee described in Chapter 4: what is on screen is always exactly what the FEN says, because it was just built from it.

The one wrinkle is animation. A naive clear-and-rebuild would make pieces teleport. So the previous positions are captured first — `snapshotBoard()` — and any piece that has moved is animated from its old coordinates to its new ones, giving the glide effect while still fundamentally being a rebuild.

JAVASCRIPT

```

function refreshUI(suppressGlide) {
  renderBoard(suppressGlide); // repaint 64 squares from currentNode.fen
  renderMoveTree();          // repaint the move list and variations
  analyzePosition();          // ask the engine about the new position
}

```

Every state change in the application funnels through this one function. There is exactly one way for the screen to change, which makes the update path trivial to reason about and impossible to partially forget.

Piece assets

JAVASCRIPT

```

function getPieceImage(color, type) {
  const names = { p:"pawn", n:"knight", b:"bishop", r:"rook", q:"queen", k:"king" };
  return `pieces/${color}_${names[type]}.png_shadow_512px.png`;
}

```

This single function is the seam through which piece theming will eventually be implemented (see Chapter 21). Because every piece image in the system is resolved here, swapping a theme

means changing one function rather than hunting through the renderer.

Drag and drop

The HTML5 drag-and-drop API is not used. It is designed for dragging data between documents, its drag image is difficult to style, and its event model is awkward. Cheese implements dragging directly with pointer events.

1. **Pointer down on a piece** — record the origin square, clone the piece into a floating 'ghost' element positioned at the cursor, and query chess.js for the legal destinations so they can be dotted.
2. **Pointer move** — move the ghost. Nothing else happens; the board itself is untouched during the drag.
3. **Pointer up** — resolve the square under the cursor with `getDragTargetSquare()`, test it against `isLegalMove()`, and either call `playMove()` or run `animateSnapBack()` to return the ghost to its origin.

IMPORTANT

Board flipping and coordinate maths. When the user plays Black in Training, the board is flipped with a CSS transform. A transform changes what the user sees but *not* the DOM geometry the hit-testing reads, so a naive `getDragTargetSquare()` returns the square that is visually opposite the one the user dropped on. The function must therefore be flip-aware and invert its file and rank calculation when the board is flipped. This bug is subtle, reproduces only when playing Black, and is the reason the glide animation is suppressed in the flipped orientation.

Feedback: highlighting, sound, and check

Signal	Trigger	Implementation
Legal-move dots	Piece selected or drag started	<code>showValidMoves()</code> asks chess.js for destinations and adds a marker class to each.
Last-move highlight	Any move applied	<code>applyLastMoveHighlight()</code> tints the origin and destination squares.
Check flash	Resulting position is check	<code>findKingSquare()</code> locates the king; <code>flashCheck()</code> runs a brief animation on it.
Sound	Any move applied	<code>moveSoundName()</code> inspects the chess.js move flags to choose between move, capture, castle, promote, and check.

Sound selection is a good example of why delegating rules to chess.js pays off. The move object returned by `move()` carries flags describing exactly what kind of move occurred — capture, castle, promotion, en passant — so choosing the correct sound requires no board inspection at all. It is a lookup on data the rules engine already computed.

CHAPTER 9

Opening Explorer

Searching the complete ECO catalogue in the browser

The Opening Explorer answers one question quickly: what is this opening called, and what are its moves? It does so over the full ECO classification, with no server and no perceptible lag.

The data

The Encyclopaedia of Chess Openings divides all of chess opening theory into five volumes, A through E, each containing a hundred numbered classifications. Cheese ships this as five static JSON files, one per volume.

JAVASCRIPT

```
const ECO_FILES = [  
  'ECO/ecoA.json',  
  'ECO/ecoB.json',  
  'ECO/ecoC.json',  
  'ECO/ecoD.json',  
  'ECO/ecoE.json',  
];
```

Each entry carries a name, an ECO code, and the move sequence that defines the line. The files are fetched once on page load, concatenated into a single in-memory array, and never fetched again. Search then operates entirely on that array — an in-memory substring filter over a few thousand records, which completes faster than a keystroke.

NOTE

Serving the ECO catalogue as static JSON rather than querying an API is the no-round-trips principle in action. The whole dataset is a few hundred kilobytes; it is cheaper to ship all of it once than to ask a server for slices of it repeatedly — and it works offline afterwards.

The rendering problem

Loading the data is easy. Rendering it is not. An empty search matches every opening in the catalogue, and constructing thousands of DOM cards in a single synchronous pass would lock the main thread for hundreds of milliseconds — the page would visibly hang on load.

The solution is chunked, demand-driven rendering.

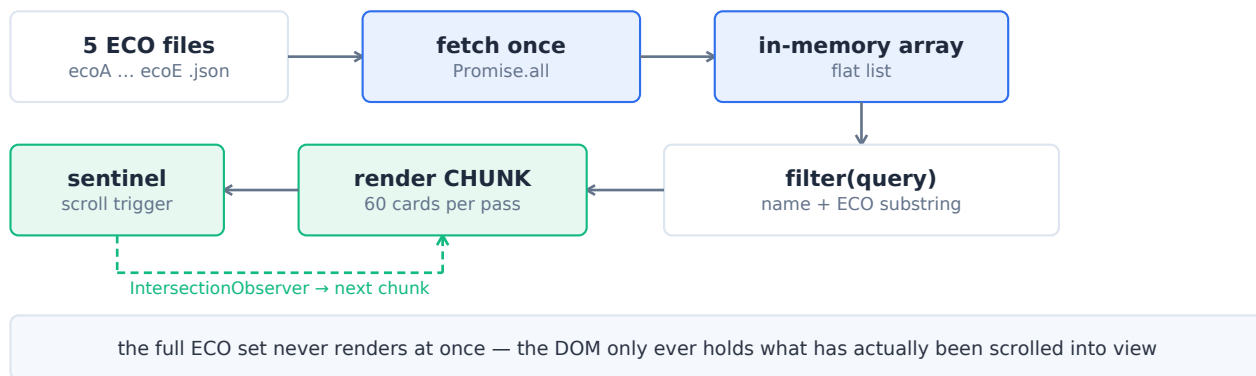


Figure 9.1 — The Explorer never renders more than the user has actually scrolled to.

Chunking and the sentinel

JAVASCRIPT

```

const CHUNK = 60; // cards added to the DOM per render pass

// A zero-height sentinel element sits at the bottom of the grid.
// When it scrolls into view, the next chunk is appended.
const io = new IntersectionObserver(entries => {
  if (entries[0].isIntersecting) renderNextChunk();
});
io.observe(sentinel);

```

Sixty cards are rendered initially. A zero-height sentinel element sits below them. When the user scrolls far enough that the sentinel enters the viewport, an `IntersectionObserver` fires and the next sixty cards are appended, which pushes the sentinel down again. The effect is infinite scroll, but the mechanism is worth appreciating: the DOM only ever contains what the user has actually looked at.

`IntersectionObserver` is used rather than a scroll event listener for a specific reason. Scroll handlers fire on the main thread, dozens of times per second, and must be throttled to avoid jank. `IntersectionObserver` is evaluated by the browser off the critical path and calls back only when the intersection state actually changes — it is both simpler and faster.

The reveal cascade

A second `IntersectionObserver` handles the entrance animation. Cards fade in as they appear, but staggered by row rather than individually, so a batch of sixty cards arrives as a handful of cascading rows rather than sixty independent flickers.

JAVASCRIPT

```

const REVEAL_STEP = 70; // ms between rows in a cascade
const REVEAL_CAP = 6; // never stagger more than 6 rows in one batch

// group revealed cards by their y-offset, then delay each row a little more
const delay = Math.min(rowIndex, REVEAL_CAP) * REVEAL_STEP;

```

The `REVEAL_CAP` is the detail that makes this work. Without it, a large batch would produce a stagger long enough that the last row appeared noticeably late — the animation would stop being a flourish and start being a wait. Capping the stagger at six rows keeps the cascade under

half a second regardless of batch size.

Hand-off to Analysis

Selecting an opening writes it to `localStorage` under `selectedOpening` and navigates to the Analysis page, which reads the key, replays the moves onto the board, and deletes the key. The one-shot pattern is described in Chapter 12.

Limitations and future work

- **No position-based lookup.** The Explorer searches by name and code, not by the position currently on the board. Detecting the opening of an arbitrary position requires indexing the ECO data by FEN or by move prefix — a worthwhile 2.0 feature.
- **No frequency statistics.** Real opening explorers show how often each move is played and how it scores. That requires a master-game database orders of magnitude larger than the eight collections currently shipped, and is a genuine case for an external API.
- **No line navigation.** An opening loads onto the board as a finished position; the user cannot step through its moves within the Explorer itself.

CHAPTER 10

Master Game Database

PGN parsing, and a registry that makes players data

The Database page holds curated collections of games from eight world-class players. Its most interesting property is not the games — it is that adding a new player requires no code.

The player registry

JAVASCRIPT

```
1 const PLAYERS = [  
2   { id: 'carlsen', name: 'Magnus Carlsen', image: 'Magnus/carlsen.jpg',  
3     pgn: 'Magnus/magnus1.pgn', match: 'carlsen' },  
4   { id: 'fischer', name: 'Bobby Fischer', image: 'Fisher/fisher.jpg',  
5     pgn: 'Fisher/fisher1.pgn', match: 'fischer' },  
6   { id: 'karpov', name: 'Anatoly Karpov', image: 'Anatoly/anatoly.jpeg',  
7     pgn: 'Anatoly/anatoly.pgn', match: 'karpov' },  
8   { id: 'botvinnik', name: 'Mikhail Botvinnik', image: 'Botvinik/botvinik.jpeg',  
9     pgn: 'Botvinik/botvinik.pgn', match: 'botvinnik' },  
10  // ... Gukesh, Nakamura, Tal, Anand  
11 ];
```

Each entry is a plain object: an identifier, a display name, a portrait path, a PGN path, and a match string. Everything the page does is driven off this array. The player grid is rendered by mapping over it; selecting a player fetches that entry's PGN; the game list is built from what comes back.

The consequence is that **adding a ninth player is a data change**: drop a portrait and a PGN file into a new folder, add one object to the array, and the page grows a new player. No rendering code is touched. This is the difference between a page that has eight players and a page that has a player system.

TIP

The match field exists because PGN header names are inconsistent across sources — the same player appears as Carlsen, Magnus, Carlsen,M, or Magnus Carlsen. Rather than normalising every source file, the registry stores a lowercase surname and does a case-insensitive substring test against the White and Black headers. Crude, but it survives real PGN files.

Splitting a multi-game PGN

A PGN file may contain hundreds of games concatenated together. They must be split before anything else can happen. The PGN specification separates games with a blank line between the previous game's moves and the next game's header block, so the split is driven off the appearance of a new [Event tag at the start of a line.

JAVASCRIPT

```
function splitPGNGames(text) {
  // split on the boundary preceding each new header block
  return text.split(/\n\s*\n(?=\n/)).filter(chunk => chunk.trim());
}

function header(pgn, key) {
  const m = pgn.match(new RegExp('\[\[' + key + '\s+"([^\"]*)"\\]'));
  return m ? m[1] : '';
}
```

Headers are then read with a small regular expression, one key at a time. This is not a full PGN parser and does not try to be: it extracts the seven headers the game list actually displays — White, Black, Event, Date, Result — and leaves the movetext untouched as an opaque string, because chess.js will parse it properly later if and when the user opens the game.

Presentation

Raw PGN headers are not presentable. Carlsen,M and 1994.11.22 are correct but ugly, so two small functions clean them up before display.

Function	Input	Output
tidyName()	Carlsen,M	M. Carlsen
prettyDate()	1994.11.22	22 Nov 1994
buildGame()	raw PGN chunk	{ pgn, title, event, date, result }

Hand-off to Analysis

Clicking a game writes **only that game's PGN** to `cheeseImportPGN` and navigates to the Analysis page. Note what is not written: the player, the collection, or any index into a list. The Analysis page receives a self-contained PGN string and has no knowledge that a Database exists — which is precisely why the same key can be used by any future feature that wants to send a game to the board.

Limitations

- **Collections are small.** Each player ships a curated file, not a complete career. Growing these is a data task, not an engineering one.
- **No cross-collection search.** There is no way to ask 'show me every Sicilian in the database'. This requires indexing games by opening at load time — feasible, and a natural pairing with the Explorer's position-lookup feature.
- **Two players were cut for a mundane reason.** Alekhine and Caruana were excluded from 1.2 solely because no suitable portrait image was available. The registry is ready for them the moment one is.

CHAPTER 11

Training Mode

Playing a complete game against the engine

Training turns the analysis board into an opponent. It is the largest single file in the project (2,068 lines) because a playable game requires far more state management than an analysis board does.

The state machine

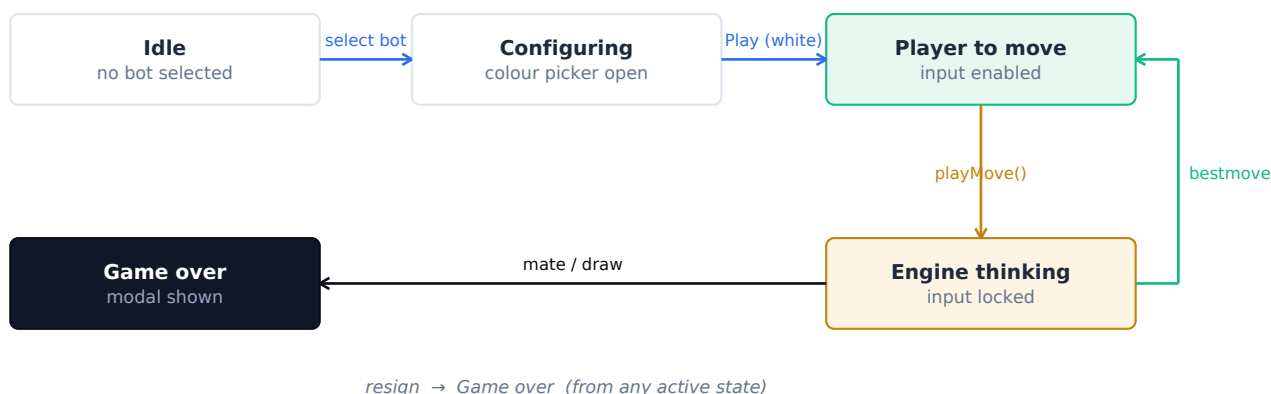


Figure 11.1 — Training mode states. Input is locked whenever it is not the player's turn.

An analysis board has no notion of whose turn it is — the user may move any piece of either colour at any time. A game does. Introducing an opponent means introducing turn ownership, and turn ownership must be enforced or the player can simply move for the engine.

Turn enforcement

Two flags carry the whole game: `trGameActive` and `trPlayerColor`. Both the click handler and the drag handler consult them before permitting any interaction.

JAVASCRIPT

```
// guard present in BOTH the click and the drag-start paths
if (!trGameActive) return;           // no game in progress
if (chess.turn() !== trPlayerColor) return; // not the player's turn
```

IMPORTANT

The guard must exist in *both* handlers. Guarding only the click path leaves the drag path open, and a user can drag the engine's pieces while it is thinking. Every input path into the board needs the same check — this is an argument for eventually funnelling all board input through a single entry point rather than two parallel ones.

The engine as an opponent

Training runs its own dedicated Stockfish worker, entirely separate from the analysis engine described in Chapter 7. The command differs too: instead of `go depth 18`, Training issues `go`

movetime 700.

The distinction matters. A depth-limited search takes as long as it takes — which varies enormously with position complexity, producing an opponent that replies instantly in simple positions and stalls for seconds in sharp ones. A time-limited search always takes 700 ms, which gives the bot a steady, human-feeling rhythm and keeps the game moving. For an opponent, consistency of response beats maximum strength.

Game termination

Detecting the end of a game correctly is harder than it looks, and Training is where Chapter 6's fresh-instance replay technique earns its keep. The live `chess.js` object has been mutated by navigation and cannot be trusted for repetition or fifty-move detection, so the terminal check replays the mainline into a clean instance and interrogates that.

Ending	Detection	Result shown
Checkmate	<code>in_checkmate()</code>	Winner is the side that did <i>not</i> just move.
Stalemate	<code>in_stalemate()</code>	Draw
Threefold repetition	<code>in_threefold_repetition()</code>	Draw
Fifty-move rule	<code>in_draw()</code>	Draw
Insufficient material	<code>insufficient_material()</code>	Draw
Resignation	User action	Opponent wins; reuses the same game-over modal with a 'won by resignation' label.

The winner-determination rule is worth stating because it is easy to invert. When checkmate is detected, the side to move is the side that is *mated* — they have no legal reply. The winner is therefore the opposite colour to `chess.turn()`, not the same one.

Board orientation

Choosing Black flips the board so the player's pieces sit at the bottom, which is the only orientation a human can actually play in. The flip is a CSS transform on the board container, with a counter-transform on each piece so the pieces themselves are not upside down.

As noted in Chapter 8, the transform is a visual operation only — the underlying DOM geometry is unchanged — so every coordinate calculation that reads pixel positions must be made flip-aware. `getDragTargetSquare()` inverts its file and rank derivation when the board is flipped. Getting this wrong produces a board where dropping a piece on d4 plays it to e5, which is as confusing as it sounds.

The no-branching rule

Training reuses the `GameNode` tree from Analysis, but a game cannot have variations — you cannot take a move back against an opponent. Training therefore guards against branching: if

currentNode is not the tip of the mainline, input is rejected and a toast explains why.

JAVASCRIPT

```
if (currentNode !== mainlineTip()) {  
  showToast("You can't branch during a game");  
  return;  
}
```

This is a good illustration of reusing a data structure across two features with different rules. The tree is general enough to model both, and the difference between 'analysis' and 'game' turns out to be a single constraint: in a game, you may only ever extend the tip.

Future work

- **Strength levels.** The bot currently plays at full engine strength, which is hopeless for the target user. Stockfish exposes `UCI_LimitStrength` and `UCI_Elo`; wiring these to the existing bot selector is the single highest-value improvement to this page.
- **More bots.** The bot panel is already built to hold several personalities; only Stockfish 18 is populated.
- **Review the finished game.** The game-over modal should offer to send the completed game straight into Analysis — the `cheeseImportPGN` mechanism already exists and would make this a small change.
- **A clock.** The player bars already render a time display; it is currently decorative.

CHAPTER 12

Persistence and the Data Model

Three keys, no database

Cheese has no server, so it has no database. Everything that survives a page reload lives in `localStorage`, and the entire persistence layer consists of three keys.

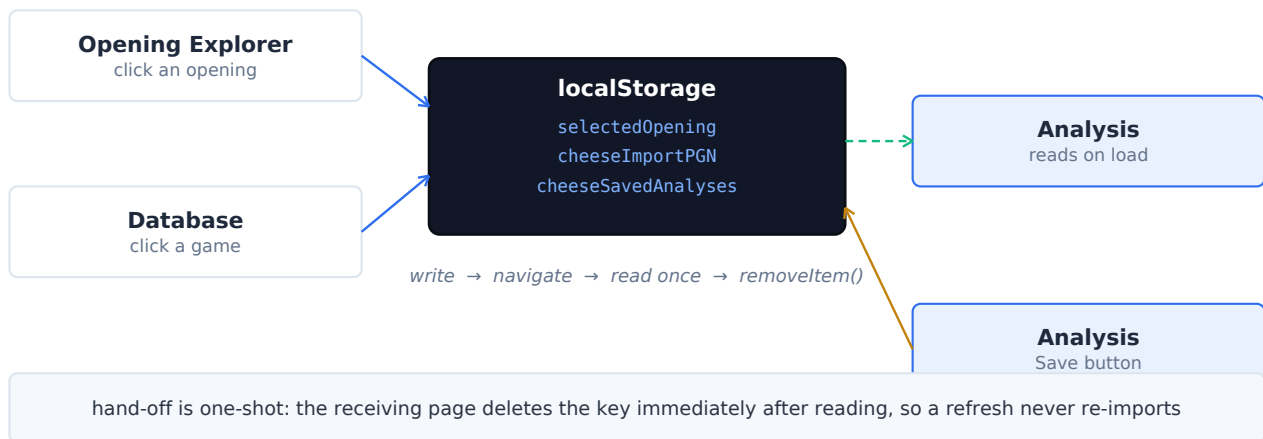


Figure 12.1 — All cross-page data flow. There is no other channel.

The three keys

Key	Written by	Read by	Lifetime
<code>cheeseSavedAnalyses</code>	Analysis (Save), Training (Save)	Analysis (Games tab)	Persistent — the user's library.
<code>cheeseImportPGN</code>	Database (game click)	Analysis (on load)	One-shot — deleted immediately after reading.
<code>selectedOpening</code>	Opening Explorer (opening click)	Analysis (on load)	One-shot — deleted immediately after reading.

The one-shot hand-off pattern

Two of the three keys are not storage at all — they are a message channel between pages. Because each page is a separate document with no shared memory, there is no way to pass an argument from the Database page to the Analysis page except through the URL or through storage. Storage was chosen because a full PGN does not belong in a query string.

JAVASCRIPT

```
1 // Sender - Database/script.js
2 localStorage.setItem('cheeseImportPGN', game.pgn);
3 window.location.href = '../Analysis/index.html';
4
5 // Receiver - Analysis/script.js, on load
6 const pgn = localStorage.getItem("cheeseImportPGN");
7 if (pgn) {
8   localStorage.removeItem("cheeseImportPGN"); // consume it IMMEDIATELY
9   importPGN(pgn);
10 }
```

IMPORTANT

Delete before you use, not after. The key is removed on the line immediately following the read, before the PGN is parsed. If it were deleted afterwards, a parse error would leave the key in storage, and the user would re-import the same broken game on every subsequent visit to the Analysis page — a bug that would appear to have nothing to do with the Database page they visited twenty minutes ago.

The saved-analysis library

`cheeseSavedAnalyses` holds a JSON array of saved analyses. Both Analysis and Training write to it — deliberately sharing one key so that a game played against the engine and a position studied on the analysis board land in the same library rather than in two disconnected lists.

JAVASCRIPT

```
1 const SAVED_ANALYSES_KEY = "cheeseSavedAnalyses";
2
3 function readSavedAnalyses() {
4   try {
5     return JSON.parse(localStorage.getItem(SAVED_ANALYSES_KEY)) || [];
6   } catch {
7     return []; // corrupt or absent -> behave as empty, never throw
8   }
9 }
10
11 function writeSavedAnalyses(list) {
12   localStorage.setItem(SAVED_ANALYSES_KEY, JSON.stringify(list));
13 }
```

The try/catch around the read is not decoration. `localStorage` is a shared, user-visible namespace; its contents can be corrupted by another script, edited by hand in devtools, or truncated by a quota failure mid-write. A parse error must degrade to an empty library, not to a page that fails to boot.

What is stored, and what is not

A saved analysis stores the PGN of the mainline plus a name and timestamp — not a serialisation of the `GameNode` tree. This is a real limitation and an honest one: **variations are lost on save**. Reloading a saved analysis replays the mainline into a fresh tree, and any sidelines the user explored are gone.

Fixing this means serialising the tree itself — straightforward in principle (the structure is a plain object graph with a parent cycle to break) but it changes the stored format, so it is scheduled

with the other storage work in 2.0.

Constraints of browser storage

Constraint	Consequence for Cheese
Roughly 5-10 MB per origin	Ample for PGN text. Thousands of games would fit; this is not a practical limit.
Strings only	Everything is <code>JSON.stringify</code> 'd on the way in and parsed on the way out.
Synchronous API	Reads and writes block the main thread. Fine at this data size; would not be at 100 MB.
Scoped to the origin	Saved analyses are invisible across browsers and devices. This is the honest cost of having no accounts.
Cleared by the user	Clearing site data destroys the library with no warning and no recovery. See Chapter 16.

CHAPTER 13

User Interface and Design System

Layout, colour, type, and the sidebar

Cheese uses no CSS framework. The visual language is hand-built and consists of a small number of repeated ideas applied consistently.

Layout

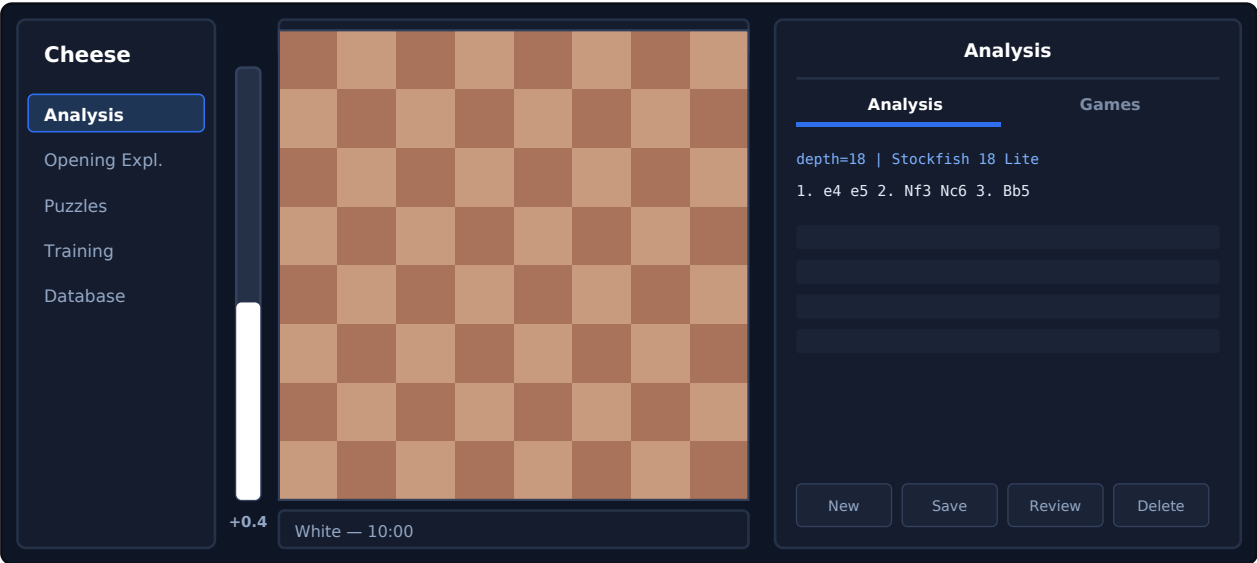


Figure 13.1 — The Analysis page. Sidebar, evaluation bar, board, and analysis panel.

Every page shares the same skeleton: a fixed 220 px sidebar on the left, and a content region that fills the remainder. The Analysis page divides that region into three columns — evaluation bar, board, and analysis panel — with the board taking as much space as it can while remaining square.

The board's squareness is load-bearing and is the reason the layout does not reflow gracefully to narrow viewports. A chessboard cannot be responsive in the usual sense: it cannot become a column, it cannot wrap, and below a certain size it stops being usable at all. This is the honest technical reason behind the mobile gate.

The design language

Element	Treatment
Background	A single illustrated scene, shared site-wide, with a dark scrim to hold contrast for white text.
Panels	Translucent dark fills with a blurred backdrop (backdrop-filter), a 1 px white border at low opacity, and a soft drop shadow.
Borders	Almost always <code>rgba(255,255,255,0.08–0.14)</code> — a white hairline at low alpha rather than a grey line, so the border picks up the background beneath it.

Element	Treatment
Text	White at full opacity for primary content; the same white at 0.55–0.75 alpha for secondary. Colour is never used to encode meaning in text.
Accent	A single green used for active state, positive evaluation, and confirmation. Red appears only for destructive actions.
Radius	Consistently rounded — 12–20 px on panels, smaller on controls.

The glassmorphism effect is achieved with layered translucency rather than image assets: a semi-transparent dark fill, a backdrop blur, and a low-alpha white border. The result reads as frosted glass over the background scene and costs three CSS properties.

Typography

Role	Face	Notes
Interface	Arial / system sans-serif	No web font is loaded. This is a deliberate performance choice — a web font is a render-blocking request for a purely cosmetic gain.
Code, FEN, PGN	Courier New / monospace	Used wherever alignment carries meaning.

TIP

Not loading a web font is the single cheapest performance decision in the project. The first paint requires no font fetch, so there is no flash of invisible text and no layout shift when a font arrives. The cost is that the interface uses a system face rather than a distinctive one — a trade worth revisiting only if the font can be self-hosted and preloaded.

The sidebar

The sidebar is identical on every page except for which item carries the active state. The active item is rendered as a bare anchor with an active class; every other item is a link.

HTML

```
<!-- the active page: not a link, visually highlighted -->
<a class="left-nav-item left-nav-active">Analysis</a>

<!-- every other page: a link -->
<li><a class="left-nav-item left-nav-item-link" href="../Training/index.html">
  Training
</a></li>
```

Because there is no templating system, this markup is **duplicated in all six pages**. Adding a nav item means editing six files, and forgetting one produces an inconsistent site. This is the clearest example of the debt that the no-build-step decision incurs, and Chapter 20 proposes the fix.

The mobile gate

The gate is pure CSS with no JavaScript. It is hidden by default, so desktop users never see it or pay for it, and it activates only inside a media query.

CSS

```
.mobile-gate { display: none; }          /* desktop: costs nothing */

@media (max-width: 768px) {
  body > *:not(.mobile-gate) { display: none !important; }
  html, body { overflow: hidden !important; }
  .mobile-gate { display: flex; position: fixed; inset: 0; z-index: 99999; }
}
```

The `body > *:not(.mobile-gate)` selector hides every top-level element *except* the gate, which leaves the page's own background visible behind the notice. The card is styled with the same glass language as the rest of the site, so even the blocked experience looks like part of the product.

IMPORTANT

A useful diagnostic: if the mobile notice ever appears on *desktop*, it means `mobile-gate.css` failed to load — the gate markup is in the DOM with no CSS to hide it. It is not a media-query bug; it is a 404. This exact symptom appeared during the first deployment and is documented in Chapter 21.

CHAPTER 14

Game Flow

What happens between the click and the evaluation

This chapter traces the complete path of a single interaction, which is the fastest way to understand how the modules described so far actually cooperate.

Session flow

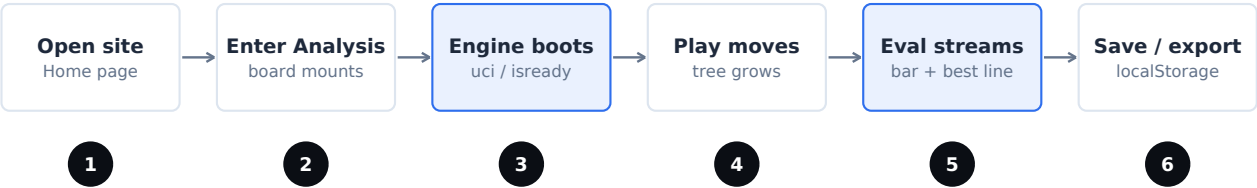


Figure 14.1 — A session, from landing to saved analysis.

The move flow

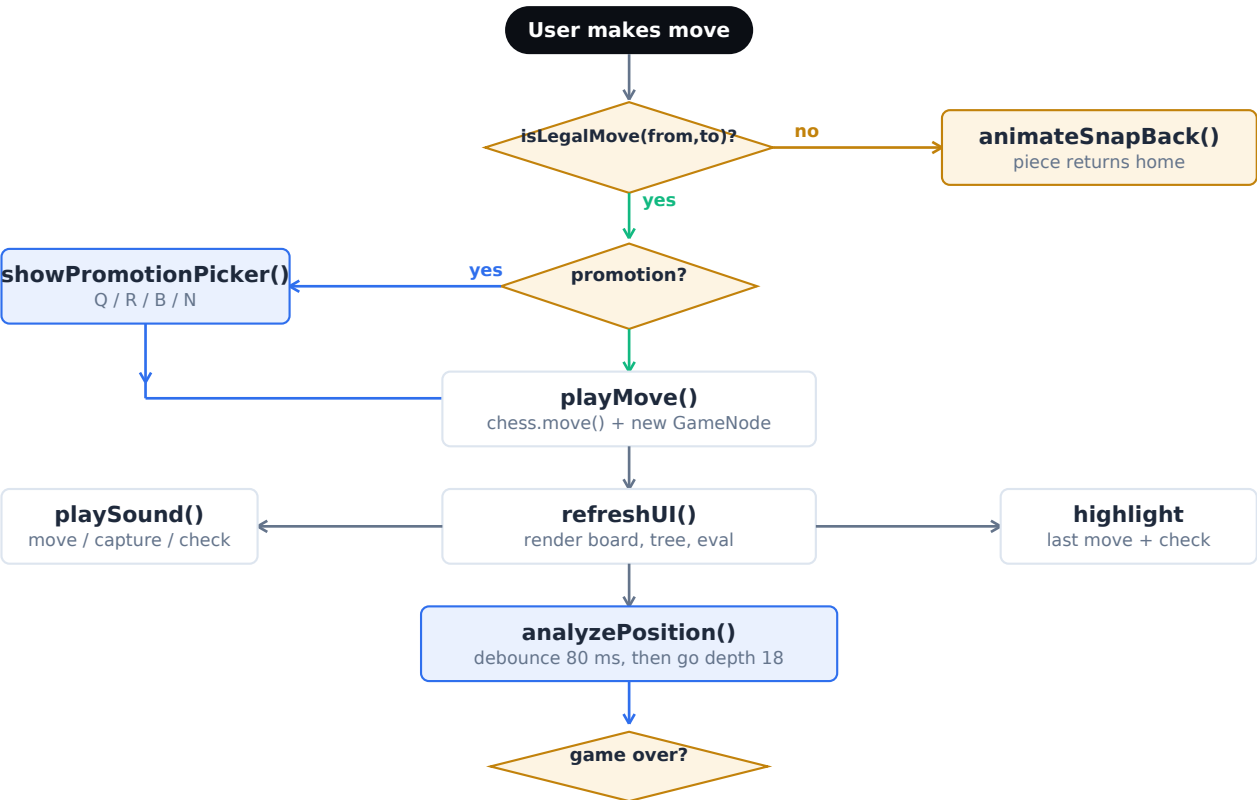


Figure 14.2 — One move, from pointer-up to engine evaluation.

Step by step

Following Figure 14.2 from the top:

1. **The user releases a piece.** The drag handler resolves which square is under the cursor, correcting for board flip if necessary.
2. **Legality is tested.** `isLegalMove(from, to)` asks `chess.js`. An illegal drop runs `animateSnapBack()` and the flow ends here — no state changes at all.
3. **Promotion is checked.** If a pawn is reaching the last rank, the move is suspended, stashed in `pendingPromotion`, and the picker opens. The flow resumes when the user chooses a piece.
4. **The move is applied.** `playMove()` hands the move to `chess.js`, which returns a move object and a new position. A new `GameNode` is created with that FEN and appended to the current node's children.
5. **Feedback fires.** The move flags select a sound; the origin and destination squares are highlighted; if the position is check, the king's square flashes.
6. **The UI repaints.** `refreshUI()` rebuilds the board from the new FEN and re-renders the move tree.
7. **The engine is asked.** `analyzePosition()` sends `stop`, waits 80 ms, then sends the new position and `go depth 18`. Evaluation streams back asynchronously and updates the bar as depth increases.
8. **Termination is checked.** If the position is mate or a draw, the game-over path runs (Training) or the engine simply reports a mate score (Analysis).

NOTE

Notice what the flow does *not* contain: no server call, no async data fetch, and no framework lifecycle. From pointer-up to a repainted board is a synchronous path through three function calls. The only asynchrony in the entire loop is the engine, and it is deliberately isolated on another thread.

The variation case

One branch of the flow deserves separate description. If the user has navigated backwards — so `currentNode` is *not* the tip of the mainline — and then plays a move, the behaviour differs by page.

Page	Behaviour	Why
Analysis	The move becomes a new child of the current node. If the node already has children, this is a second child and renders as a variation.	This is the entire point of the tree. Exploring alternatives must not destroy the original line.
Training	The move is rejected and a toast appears.	You cannot take back a move against an opponent. A game may only extend its tip.

The same data structure and the same input handlers serve both, separated only by a single guard. That is the payoff of having chosen the more general structure at the start.

CHAPTER 15

Performance

Where the time goes, and what was done about it

Cheese is fast, but not accidentally. Three specific decisions account for most of it, and one unavoidable cost dominates everything else.

The cost model

Operation	Approximate cost	Notes
Engine boot (WASM instantiate)	Hundreds of ms	Paid once per page load. The dominant cost in the system.
Board repaint (64 squares)	Sub-frame	Full rebuild, no diffing. Not worth optimising.
Move tree render	Sub-frame for typical games	Recursive; grows with the number of variations.
chess.js move + legality	Negligible	Microseconds.
ECO search over a few thousand records	Negligible	In-memory substring filter; faster than a keystroke.
Rendering the full ECO catalogue at once	Hundreds of ms — avoided	This is the cost that chunking exists to prevent.
Stockfish search (depth 18)	Hundreds of ms to seconds	Off the main thread; never blocks the UI.

What was actually optimised

1. The engine is on another thread

This is not a micro-optimisation; it is the difference between a usable application and an unusable one. A depth-18 search on the main thread would freeze the page for the duration of every search. Moving it to a Worker means the expensive operation in the system is also the one that cannot affect perceived performance at all.

2. Analysis requests are debounced

The 80 ms debounce in `analyzePosition()` converts a burst of position changes into a single search. Without it, clicking quickly through a 40-move game would start and abort 40 searches, each paying start-up cost and producing nothing. With it, one search runs, on the position the user stopped at.

3. The DOM only holds what has been scrolled to

The chunked ECO renderer (Chapter 9) is the largest single performance win in the codebase. The naive implementation — render every matching opening — produces a page that visibly hangs on load. Rendering 60 at a time and appending more on scroll keeps the DOM small and the frame rate at 60.

What was deliberately not optimised

The board repaint is a full rebuild on every single move: clear 64 squares, re-place up to 32 pieces. A virtual DOM or a diffing layer would avoid most of that work.

It was not written, and should not be. The board is 64 elements. A full repaint completes comfortably within a frame, and the alternative — tracking which squares changed and mutating only those — reintroduces exactly the class of desynchronisation bug that rebuilding-from-FEN eliminates. The correct trade is to spend the microseconds and keep the invariant.

TIP

This is the general principle: optimise the things that are actually slow (WebAssembly boot, rendering thousands of nodes) and refuse to optimise the things that merely *look* wasteful (repainting 64 divs). Correctness invariants are worth more than microseconds.

Memory

- **The engine dominates.** The Stockfish WASM module and its hash table account for the large majority of the tab's memory footprint.
- **The move tree is small.** Each node holds a move object, a FEN string, and pointers. A long game with many variations is a few hundred nodes — kilobytes.
- **The ECO array is held permanently.** A few thousand records stay in memory for the lifetime of the Explorer page. This is intentional: re-fetching would be slower and pointless.
- **Two engines on two pages, never at once.** Because pages are separate documents, navigating from Analysis to Training tears down the first engine entirely. There is never more than one Stockfish instance alive.

Loading

The site loads quickly for unglamorous reasons: there is no framework bundle to parse, no web font to fetch, no build output, and no API round-trip before first paint. The board is interactive as soon as the HTML and CSS land; the engine finishes booting shortly afterwards and the evaluation bar populates.

The single heaviest asset is the Stockfish WASM binary. It is fetched once and then served from the browser cache, and Cloudflare's edge cache means it usually arrives from a nearby location rather than from origin.

Future optimisation

- **Multi-threaded Stockfish.** Serving C00P and C0EP headers unlocks SharedArrayBuffer and therefore multi-threaded search — several times the strength at the same wall-clock time. The largest available win.
- **Cache evaluations on the node.** `GameNode.engineEval` already exists as a field. Populating it and reading from it on navigation would make stepping backwards through an analysed game instantaneous instead of re-searching every position.
- **A service worker.** Caching the engine binary and the ECO files would make the site work fully offline after first visit.

CHAPTER 16

Security and Privacy

A small attack surface, honestly assessed

Cheese has an unusually small attack surface, for a structural reason: it has no server, no accounts, no user-generated content that any other user can see, and no data worth stealing. Most of the standard web threat model simply does not apply.

The threat model

Threat	Applicability	Reason
SQL injection	None	There is no database and no query.
Server-side RCE	None	There is no server-side execution.
Authentication bypass	None	There is no authentication.
Session hijacking	None	There are no sessions and no cookies.
CSRF	None	There are no state-changing requests to forge.
Data breach	None at rest	No user data is transmitted or stored anywhere but the user's own browser.
XSS	Applies	The one real category. User-supplied PGN text reaches the DOM.
Supply chain	Applies	chess.js is loaded from a third-party CDN.
Denial of service	Local only	Malformed input can hang the user's own tab; it affects nobody else.

The one real risk: PGN as untrusted input

PGN is the only untrusted input the application accepts. It arrives by paste, and it flows into the parser, into headers that are displayed, and into the move tree that is rendered. It must be treated as hostile.

The important rule is that **parsed values must never be written to** `innerHTML`. A PGN header is an arbitrary string chosen by whoever wrote the file, and a header such as the following is entirely legal PGN:

TEXT

```
[Event "<img src=x onerror=alert(document.cookie)>"]
```

Written to `innerHTML`, that executes. Written to `textContent`, it renders as literal text and is inert. The mitigation is therefore mechanical rather than clever:

JAVASCRIPT

```
// UNSAFE - never do this with parsed PGN data
el.innerHTML = header(pgn, 'Event');

// SAFE - the browser treats the value as text, never as markup
el.textContent = header(pgn, 'Event');
```

IMPORTANT

Audit note. This is the highest-value security review the project can receive: walk every path from parsed PGN to the DOM in Database/script.js and Analysis/script.js, and confirm that every one terminates in textContent or in a created text node. Because there are no accounts and no cookies worth stealing, the practical impact of an XSS here is low — but 'low impact' is not a reason to leave the hole open, and a future version with accounts would make it serious overnight.

Input validation

- **Move legality** is validated by chess.js, which is the authority. No move enters the tree without passing through it.
- **PGN structure** is validated by chess.js on import. A malformed PGN fails to load rather than producing a corrupt tree.
- **FEN strings** are never accepted from the user directly in 1.2 — they are only ever produced by chess.js. A future FEN-paste feature would need explicit validation, because a malformed FEN is a straightforward way to crash the engine worker.
- **Storage contents** are parsed inside try/catch and degrade to empty on failure (Chapter 12).

Supply chain

chess.js is loaded from a public CDN by script tag. This is a genuine, if small, supply-chain exposure: a compromised CDN could serve modified JavaScript with full access to the page.

Two mitigations are available and neither is yet applied. **Subresource Integrity** adds a cryptographic hash to the script tag, so the browser refuses to execute the file if its contents change. **Self-hosting** removes the third-party dependency entirely — the library is small and Cheese already self-hosts a much larger WASM binary. Self-hosting is the recommended fix and is on the 2.0 list.

HTML

```
<!-- current -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/chess.js/0.10.3/chess.min.js"></script>

<!-- better: integrity check -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/chess.js/0.10.3/chess.min.js"
        integrity="sha384-..." crossorigin="anonymous"></script>

<!-- best: no third party at all -->
<script src="lib/chess.min.js"></script>
```

Privacy

Privacy is the strongest property Cheese has, and it is a consequence of the architecture rather than a policy.

- **No analytics or telemetry** of any kind.
- **No cookies**, and therefore no consent banner.
- **Positions never leave the device.** The engine runs locally, so the games a user analyses are never transmitted anywhere — an assurance that no server-side analysis platform can offer, regardless of its privacy policy.
- **The only outbound request** after page load is the chess.js CDN fetch.

The data-loss risk

IMPORTANT

The most likely way a user is actually harmed by Cheese is not an attack — it is **losing their saved analyses**. `localStorage` is destroyed by clearing site data, by some privacy modes, and silently by browser eviction under storage pressure. There is no backup and no recovery. Version 1.2 does not warn the user about this, and it should. An export-library-to-file button is the cheapest possible insurance and is a priority roadmap item.

CHAPTER 17

Accessibility

An honest audit, including what is missing

This chapter is deliberately written as a gap analysis rather than a feature list, because version 1.2's accessibility is weak and describing it any other way would be dishonest.

Current state

Dimension	Status	Detail
Colour contrast	Good	White text on dark translucent panels over a darkened background. The scrim behind every panel exists specifically to guarantee contrast against the illustrated backdrop.
Motion	Reasonable	Animations are short and purposeful — piece glide, card reveal, check flash. None loop or pulse indefinitely.
Semantic structure	Partial	Navigation uses real lists and anchors. The board, however, is 64 <code><div></code> elements with no semantic role.
Keyboard navigation	Missing	Moves cannot be made or navigated by keyboard. There is no arrow-key binding for stepping through a game.
Screen reader support	Missing	No ARIA roles, no labels on squares or pieces, and no live region announcing moves or evaluation changes.
Focus management	Missing	Modals (promotion picker, game over) do not trap focus or return it on close.
Touch support	Blocked	Touch input is not implemented; the mobile gate prevents the experience entirely.
Reduced motion	Missing	<code>prefers-reduced-motion</code> is not honoured.

NOTE

A chess interface is not inherently inaccessible — chess is a game of discrete, nameable states, and a screen-reader user can play it perfectly well if the interface announces positions in algebraic notation. The gap here is a matter of work not yet done, not of a fundamental barrier. That makes it worth fixing rather than worth excusing.

The remediation plan

The following changes are listed in order of value-per-unit-of-effort. The first three are individually small and would together transform the situation.

1. Keyboard navigation of the move list

Bind the left and right arrow keys to the existing previous/next navigation functions, and home/end to first/last. The functions already exist and are already wired to buttons; this is a

listener and four lines of dispatch. It benefits every user, not only those who need it — stepping through a game with arrow keys is simply faster than clicking.

JAVASCRIPT

```
document.addEventListener("keydown", (e) => {
  if (e.key === "ArrowLeft") { goToPreviousMove(); }
  if (e.key === "ArrowRight") { goToNextMove(); }
  if (e.key === "Home") { goToStart(); }
  if (e.key === "End") { goToEnd(); }
});
```

2. Label the board

Give every square an accessible name and every piece a description. A square is already identified internally by its algebraic name, so the label is available at zero cost — it simply is not being exposed.

HTML

```
<div class="square" id="e4"
      role="gridcell"
      aria-label="e4, white pawn">
</div>
```

3. Announce moves and evaluation

Add an `aria-live="polite"` region that receives the SAN of each move as it is played, and the evaluation when it settles. This single element is what turns the board from silent to narrated.

4. Keyboard piece movement

The largest of the four. Allow a square to be focused, selected with Enter, and a destination chosen with the arrow keys and Enter again. The legal-move data needed to validate this already exists — `showValidMoves()` computes it for the dots.

5. Honour prefers-reduced-motion

CSS

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

Three lines, and it respects an explicit operating-system-level request from the user. There is no reason not to ship it.

Why this matters for the roadmap

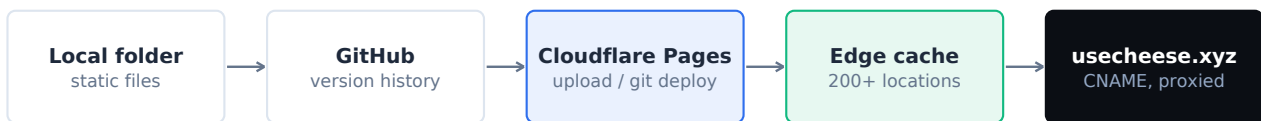
The accessibility work and the mobile work are the same work. Both require abandoning the assumption of a mouse on a wide screen; both require the board to accept input through more than one channel. Doing them together — rather than bolting touch support onto a mouse-only board and then bolting keyboard support onto that — is substantially less total effort and produces a better result.

CHAPTER 18

Deployment

Cloudflare Pages, DNS, and the traps along the way

Cheese deploys as a folder of static files. That should make deployment trivial, and eventually it is — but the first launch surfaced four distinct failure modes that are worth recording precisely, because every one of them is a trap that static sites fall into.



he note: the *.pages.dev preview URL bypasses the edge cache, so a fresh deploy shows there first; the proxied custom domain may serve a cached copy until it is p

Figure 18.1 — The deployment path, and where caching bites.

Hosting model

The site is hosted on **Cloudflare Pages** with the following configuration:

Setting	Value	Why
Framework preset	None	There is no framework.
Build command	(empty)	There is no build step.
Output directory	The repository root	Must be the directory that <i>directly</i> contains <code>index.html</code> and all page folders.
Custom domain	usecheese.xyz	A proxied CNAME to the *.pages.dev target.

Trap 1: Workers is not Pages

IMPORTANT

Cloudflare offers both **Workers** and **Pages**, and the dashboard presents them together. Workers is a serverless *function* platform; Pages is a static *site* platform. Deploying a folder of HTML as a Worker produces a project that serves essentially nothing — the symptom is that the landing page appears to work while every other route 404s. The fix is not a configuration change; it is to redeploy the project as Pages.

Trap 2: the output directory must be the true root

If the uploaded folder is `project/` and `index.html` sits *inside* it, then the deployed site root becomes `/project/` and the domain root serves nothing. When uploading, the contents of the project folder must be selected — not the folder itself.

Trap 3: case sensitivity

This is the trap that produces the most confusing failures, because the site works perfectly on the developer's machine and breaks only in production.

Windows and macOS use case-insensitive filesystems. A page that links to `pieces/w_pawn.png` will happily load a file actually named `Pieces/W_Pawn.png`. Cloudflare's servers run Linux, which is case-sensitive: the same link 404s. The symptom during the 1.2 launch was a board that rendered with no pieces at all — the squares were there, the images were not.

IMPORTANT

Git compounds this. If a directory's *capitalisation* is changed locally, Git often does not notice, and the old casing stays in the repository — so the fix appears to be committed but production does not change. Force it explicitly:

```
git mv OldName tempname && git mv tempname newname
```

Trap 4: two different caches

After a successful deploy, the new version appears immediately on the `*.pages.dev` preview URL but the custom domain continues to serve the old one. This looks like a broken deployment and is not.

The custom domain is **proxied** through Cloudflare's CDN — the orange cloud — so responses are cached at edge locations worldwide. The `pages.dev` URL is not cached the same way. The deploy succeeded; the edge is simply still serving what it already had.

Symptom	Action
Old version on the custom domain, new version on <code>pages.dev</code>	Cloudflare dashboard → the domain → Caching → Purge Everything .
Old version even after purging	The browser's own cache. Hard-refresh (Ctrl+Shift+R) or use a private window.

DNS

The domain is registered with Cloudflare, so DNS lives in the same account. The record is a single proxied CNAME:

TEXT					
Type	Name	Content	Proxy	TTL	
CNAME	usecheese.xyz	cheese-website-55r.pages.dev	Proxied	Auto	

Two notes. The target is a bare hostname — no `https://` and no trailing slash; pasting a full URL is rejected. And when the custom domain is attached through the Pages interface, Cloudflare usually creates this record automatically, so the manual step is often unnecessary — an 'a record

with that host already exists' error generally means it has already been done for you.

The deployment checklist

For every release:

1. Confirm the project root contains `index.html` directly, alongside all page directories.
2. Confirm every directory and file name matches the links that point at it, **including capitalisation**.
3. Deploy to Pages (upload the folder's contents, or push to the connected Git repo).
4. Test on the `*.pages.dev` URL first — it is uncached and shows the truth.
5. Click every page. Open devtools → Network and confirm no 404s, particularly for `mobile-gate.css`, the piece images, and the engine binary.
6. Purge the Cloudflare cache.
7. Hard-refresh the custom domain and verify.

Recommended improvement: connect Git

Version 1.2 was deployed by drag-and-drop upload. Connecting the Cloudflare Pages project to the GitHub repository instead makes deployment a `git push`, eliminates the possibility of uploading the wrong folder level, and gives every deployment a commit to roll back to. There is no reason not to do this.

CHAPTER 19

Developer Guide

Running, modifying, and extending Cheese

Prerequisites

None, beyond a browser and a text editor. There is no Node installation, no package manager, no dependency tree, and no build tool. This is the practical payoff of the no-build-step decision: the barrier to a first contribution is opening a file.

Getting the code

BASH

```
git clone https://github.com/<your-account>/cheese.git
cd cheese
```

Running locally

The site **must be served over HTTP**. Opening `index.html` directly from the filesystem (`file://`) will not work, because Web Workers and `fetch()` are both blocked under the `file://` origin. The engine will fail to start and the ECO files will fail to load.

BASH

```
# any static server works - Python needs no installation on most systems
python3 -m http.server 8000

# then open http://localhost:8000
```

TIP

If the board renders but the evaluation bar never populates, and the console shows a worker or CORS error, the cause is almost always that the page was opened as a file rather than served. Start a static server.

Common tasks

Add a player to the Database

1. Create a directory under `Database/` for the player.
2. Add a portrait image and a multi-game `.pgn` file to it.
3. Add one entry to the `PLAYERS` array in `Database/script.js`.

JAVASCRIPT

```
{
  id:    'alekhine',
  name:  'Alexander Alekhine',
  image: 'Alekhine/alekhine.jpg',
  pgn:   'Alekhine/alekhine.pgn',
  match: 'alekhine'           // lowercase surname, matched against PGN headers
}
```

No rendering code changes. The `match` field must be the lowercase surname as it appears inside the PGN's White and Black headers — it is compared as a case-insensitive substring.

Add openings to the Explorer

Openings live in the five ECO JSON files under `opening-explorer/ECO/`. Adding an entry to any of them makes it searchable immediately; the file is loaded and filtered generically, so no code changes are required. To add a new *volume* file, extend the `ECO_FILES` array.

Replace or upgrade Stockfish

1. Obtain a WASM Stockfish build (a `.js` glue file plus a `.wasm` binary).
2. Replace the contents of `Analysis/stockfish/` (and `Training/stockfish/`, which has its own copy).
3. Confirm the worker entry point is still named `stockfish.js`, or update the new `Worker(...)` path.
4. Update the `STOCKFISH_VERSION` constant, which is displayed in the analysis panel.

Because the integration speaks plain UCI and uses no engine-specific features, any UCI-compliant engine compiled to WASM will work — not only Stockfish.

Add a theme (not yet supported)

Theming is not implemented, but the seam exists. Every piece image in the system is resolved through `getPieceImage(color, type)`, and the board squares are two SVG files referenced from CSS. A theme system would:

- Replace the hard-coded path in `getPieceImage()` with a lookup into a theme object.
- Move the board square colours into CSS custom properties so they can be swapped at runtime.
- Persist the chosen theme in `localStorage` under a fourth key.

Code conventions

Convention	Rule
Rules logic	Never implement chess rules. Ask <code>chess.js</code> .
Rendering	Never mutate game state inside a render function. Render from the FEN; change state elsewhere.
Repaints	Always go through <code>refreshUI()</code> . Do not call <code>renderBoard()</code> directly unless you have a specific reason and know why.
Storage	Wrap every read in <code>try/catch</code> ; degrade to empty, never throw.
Engine	Always send stop before a new position.
Naming	New directories are lowercase-hyphenated. No spaces, ever.

Testing

There is no automated test suite. This is the most significant gap in the engineering process, and it is stated plainly rather than hidden: every change to Cheese is currently validated by playing with it.

The pragmatic first step is not a full framework but a handful of assertions around the pure logic that already has no DOM dependency — `generatePGNFromNode()`, the tree navigation helpers,

`splitPGNGames()`, and the header parsers. These are pure functions with clear inputs and outputs, they contain the logic most likely to break silently, and they could be covered in an afternoon.

Until then, the manual regression checklist before any release:

- Play a move; confirm the sound, the highlight, and the evaluation update.
- Navigate backwards and play a different move; confirm a variation appears and the original line survives.
- Promote a pawn; confirm the picker appears and all four pieces work.
- Castle both sides; confirm the rook moves and the castle sound plays.
- Import a PGN from the Database; confirm it loads and the key does not re-import on refresh.
- Play a full Training game to checkmate; confirm the correct winner is announced.
- Resign; confirm the modal reports resignation.
- Save an analysis, reload the page, and restore it.
- Load the site at a narrow viewport; confirm the mobile gate appears.
- Load the site on desktop; confirm the mobile gate does *not* appear.

CHAPTER 20

Internal API Reference

Every module, documented as though it were public

Cheese has no exported API — the scripts run in page scope. This chapter documents the principal functions anyway, because a future contributor needs to know what exists before they can safely change anything.

20.1 Core state and rendering

refreshUI(suppressGlide)

JAVASCRIPT

```
function refreshUI(suppressGlide: boolean): void
```

The single repaint entry point. Every state change in the application ends here. Rebuilds the board from `currentNode.fen`, re-renders the move tree, and requests a fresh engine evaluation.

Parameters

`suppressGlide` — when true, pieces are placed without the movement animation. Used when jumping to a distant position, where animating would be nonsensical, and when the board is flipped.

Returns

`void`

Side effects

Mutates the DOM. Posts messages to the engine worker.

JAVASCRIPT

```
// after any change to currentNode:  
currentNode = someNode;  
refreshUI(true); // no glide when jumping
```

renderBoard(suppressGlide)

JAVASCRIPT

```
function renderBoard(suppressGlide: boolean): void
```

Repaints all 64 squares from the current node's FEN. Snapshots the previous piece positions first so that moved pieces can be animated from their old coordinates.

Parameters

`suppressGlide` — skip the animation.

Returns

`void`

Side effects

Clears and rebuilds the board DOM. Reapplies last-move highlighting and the check flash.

playMove(moveInput, suppressGlide, suppressSound)

JAVASCRIPT

```
function playMove(moveInput: string | object,
  suppressGlide?: boolean,
  suppressSound?: boolean): GameNode | null
```

Applies a move, creates the resulting `GameNode`, appends it to the current node's children, advances `currentNode`, and repaints. This is the only sanctioned way to add a move to the tree.

Parameters

`moveInput` — SAN string or a {from, to, promotion} object, as accepted by chess.js.
`suppressGlide`, `suppressSound` — used during bulk replay (PGN import) so that loading a 40-move game does not fire 40 sounds.

Returns

The new `GameNode`, or null if the move was illegal.

Side effects

Mutates the tree and `currentNode`. Plays a sound. Triggers a full repaint and a new engine search.

JAVASCRIPT

```
// replay a PGN silently, then show the final position
for (const san of moves) playMove(san, true, true);
refreshUI(true);
```

20.2 The move tree

GameNode

JAVASCRIPT

```
class GameNode {
  constructor({ move, fen, parent })
  addChild(node: GameNode): GameNode
  findChildBySAN(san: string): GameNode | undefined
  getPath(): GameNode[]
}
```

A single position in the analysis tree. See Chapter 6 for the design rationale.

Parameters

`move` — the chess.js move object that produced this position (null at the root).
`fen` — the position *after* the move. The source of truth for rendering.
`parent` — parent node, null at the root.

Returns

`getPath()` returns the root-first array of nodes leading to this one.

Side effects

None. The structure is inert data and holds no DOM references.

mainlineTip()

JAVASCRIPT

```
function mainlineTip(): GameNode
```

Walks `children[0]` from the root until it reaches a node with no children. Used by Training to test whether the user is at the end of the game (branching is forbidden there).

Parameters	None.
Returns	The final node of the mainline.
Side effects	None — a pure traversal.

JAVASCRIPT

```
if (currentNode !== mainlineTip()) {
  showToast("You can't branch during a game");
  return;
}
```

generatePGNFromNode(node)**JAVASCRIPT**

```
function generatePGNFromNode(node: GameNode): string
```

Produces standard PGN movetext for the line from the root to the given node, by walking parent pointers and reversing.

Parameters	node — the node to generate up to. Usually <code>currentNode</code> or <code>mainlineTip()</code> .
Returns	A PGN movetext string with move numbers.
Side effects	None — pure.

JAVASCRIPT

```
// current limitation: only the mainline to `node` is emitted.
// Variations are NOT included in the export.
const pgn = generatePGNFromNode(mainlineTip());
```

20.3 Engine

analyzePosition()**JAVASCRIPT**

```
function analyzePosition(): void
```

Requests an engine evaluation of the current node's position. Sends `stop` immediately, then debounces the actual search request by 80 ms.

Parameters	None — reads <code>currentNode.fen</code> .
Returns	<code>void</code> . Results arrive asynchronously via <code>engine.onmessage</code> .
Side effects	Posts <code>stop</code> , <code>position fen</code> , and <code>go depth 18</code> to the worker. Cancels any pending debounce timer.

updateEvalBar(evalValue)

JAVASCRIPT

```
function updateEvalBar(evalValue: number | string): void
```

Maps a white-relative evaluation in pawns to the height of the evaluation bar. Handles the terminal cases (checkmate on the board) and clamps extreme values.

Parameters	evalValue — white-relative score in pawns (already normalised and divided by 100).
Returns	void
Side effects	Mutates the eval bar's height and label.

20.4 Persistence

readSavedAnalyses() / writeSavedAnalyses(list)

JAVASCRIPT

```
function readSavedAnalyses(): Analysis[]
function writeSavedAnalyses(list: Analysis[]): void
```

Read and write the saved-analysis library under the `cheeseSavedAnalyses` key. The read is defensive: corrupt or absent data yields an empty array rather than an exception.

Parameters	list — the complete library. The write replaces, it does not merge.
Returns	readSavedAnalyses() returns an array, always.
Side effects	Touches localStorage. Both Analysis and Training write to this same key by design.

20.5 Known duplication

The following logic is duplicated between `Analysis/script.js` and `Training/script.js` rather than shared. This is recorded here as debt, not as design.

Duplicated	Consequence
Board rendering and drag-and-drop	A rendering fix must be applied twice, and the Training copy has diverged (it is flip-aware; the Analysis copy is not).
The <code>GameNode</code> class	Two definitions that must be kept in step by hand.
Sidebar markup	Present in all six HTML files. Adding a nav item means editing six files.
Sound loading and selection	Two copies of the same sound map.

TIP

The remedy is a shared `lib/` directory holding `board.js`, `tree.js`, and `engine.js`, imported by both pages as ES modules. ES modules work natively in the browser with no build step, so this refactor costs nothing architecturally — it is simply work that has not been done yet. It is the top engineering item on the 2.0 list.

CHAPTER 21

Technical Challenges and Lessons Learned

The bugs that taught something

Every non-trivial decision in this codebase has a bug behind it. This chapter records the ones worth remembering — what broke, why, what fixed it, and what it cost.

Engine and state

1. The evaluation bar pointed the wrong way

Symptom	The bar swung to the opposite side on alternate moves, as though the player winning changed every ply.
Root cause	UCI reports scores relative to the side to move , not relative to White. A score of +150 with Black to move means Black is better. The raw value was being fed straight into a bar that assumed White's perspective.
Fix	Normalise on receipt: construct a throwaway chess.js instance from the FEN, check turn(), and negate the score if it is Black's move.
Lesson	<i>Protocols have conventions that are obvious to their authors and invisible to everyone else. Read the specification for the field you are parsing, not just its name.</i>

2. The engine reported evaluations for the wrong position

Symptom	After clicking quickly through a game, the evaluation shown belonged to a position several moves earlier.
Root cause	A new position command was being sent while the engine was still searching the previous one. UCI engines are stateful, and the results already in flight continued to arrive and overwrite the display.
Fix	Send an explicit stop synchronously , before the debounce timer, so the engine is guaranteed idle before the new position arrives.
Lesson	<i>When talking to a stateful process, make the state transition explicit. Do not assume a new command implicitly cancels the old one.</i>

3. The training bot sometimes never moved

Symptom	During a game against Stockfish, the engine would occasionally simply not reply, leaving the game hung.
Root cause	Training and Analysis were sharing a single engine worker. The analysis debounce timer was firing during the bot's search and issuing a stop — killing the bot's own search before it produced a bestmove.
Fix	Give Training its own dedicated Worker. Two engines, two independent search lifecycles, zero interference.
Lesson	<i>Two features with different lifecycles should not share a stateful resource, even when it seems wasteful to duplicate it. The duplication is cheaper than the coupling.</i>

4. Threefold repetition was detected incorrectly

Symptom	Draws were being declared in games where no repetition had occurred.
Root cause	The live chess.js instance is mutated during navigation and variation exploration. Its internal move history — which repetition and fifty-move detection depend on — had absorbed moves from lines that were never actually played.
Fix	Detect terminal conditions by replaying the mainline into a fresh chess.js instance, and interrogate that.
Lesson	<i>If a library's answer depends on its internal history, and you have been mutating it, its answer is about your mutations — not about the game.</i>

Interface

5. Dropping a piece on Black's board played the wrong square

Symptom	When playing as Black (board flipped), dropping a piece on d4 would play the move to e5.
Root cause	The flip is a CSS transform. Transforms change what is <i>painted</i> but not the underlying DOM geometry that hit-testing reads, so the coordinate maths was computing the unflipped square.
Fix	Make getDragTargetSquare() flip-aware: invert the file and rank derivation when the board is flipped.
Lesson	<i>A visual transform is not a coordinate transform. Anything that reads pixel positions must be told about it explicitly.</i>

6. The 'mobile only' notice appeared on desktop

Symptom	The 'Mobile Support Coming Soon' card rendered on a 1440 px desktop display, where it should have been impossible.
Root cause	Not a media-query bug. mobile-gate.css had 404'd. The gate's markup sits in the DOM of every page and is hidden by display: none in that stylesheet — with the stylesheet missing, there was nothing to hide it.
Fix	Fix the path so the file actually loads.
Lesson	<i>When something styled appears unstyled, suspect the stylesheet before suspecting the logic. 'Impossible' rendering is usually a 404.</i>

Deployment

7. Only the home page worked in production

Symptom	After the first deploy, the landing page loaded and every other route returned 'page not found'.
Root cause	The project had been deployed as a Cloudflare Worker — a serverless function platform — rather than as Cloudflare Pages , which is the static-site platform. A Worker does not serve a directory tree.

Fix	Redeploy as a Pages project, with the output directory set to the true repository root.
Lesson	<i>Read what the platform actually is, not what its name suggests. The dashboards sit side by side and the distinction is not signposted.</i>

8. The board rendered with no pieces

Symptom	In production, the squares appeared but every piece image was missing. Locally, everything worked.
Root cause	Filesystem case sensitivity. The developer machine (case-insensitive) happily resolved pieces/ against a directory whose real name differed in capitalisation. Cloudflare's Linux servers did not.
Fix	Align the directory names with the links exactly, and force the rename through Git with a two-step <code>git mv</code> so the capitalisation change is actually recorded.
Lesson	<i>Your development filesystem is lying to you. Case sensitivity bugs are invisible until deployment, by construction.</i>

9. A successful deploy changed nothing

Symptom	After deploying a fix, the *.pages.dev URL showed the new version but the custom domain kept serving the old one.
Root cause	The custom domain is proxied through Cloudflare's CDN and was serving a cached copy. The preview URL is not cached the same way. The deploy had succeeded all along.
Fix	Purge the Cloudflare cache; hard-refresh to defeat the browser cache as well.
Lesson	<i>There are at least two caches between your code and your user. When a deploy 'does not work', prove where it stopped before changing anything.</i>

What the project would do differently

- **Extract shared modules from the start.** The duplication between Analysis and Training was cheap to create and is now expensive to unwind. ES modules would have cost nothing at the beginning.
- **Write the pure-function tests early.** PGN generation and tree navigation are exactly the kind of logic that breaks silently, and they are trivially testable.
- **Adopt lowercase-hyphenated paths on day one.** Two of the nine bugs above are downstream of a directory named with a capital letter and a space.
- **Deploy in week one, not at the end.** Every deployment bug in this chapter would have been found in an afternoon if the site had been on Cloudflare from the first commit, instead of arriving all at once during launch.

CHAPTER 22

Development Roadmap

From 1.2 to 4.0

The roadmap below is ordered by version. The guiding constraint — no backend — holds through 2.0 and is consciously traded away at 3.0 in exchange for cross-device sync. That trade is the single most consequential decision on this list.

Shipped

Version	Delivered
1.0	Analysis board, Stockfish evaluation, move list, PGN import.
1.1	Opening Explorer, saved analyses, PGN export.
1.2	Training mode, master-game Database, Puzzles placeholder, mobile gate, public launch on usecheese.xyz.

Version 1.3 — Puzzles and repayment

The next release should finish what 1.2 promised and pay down the debt it accumulated, rather than adding new surface area.

Item	Priority	Notes
Puzzle system	High	The placeholder page already exists and is linked from Home. A tactics trainer needs a puzzle set (FEN + solution line), a validation loop, and progress tracking in localStorage. No new architecture required.
Keyboard navigation	High	Arrow keys for move stepping. Small change, large benefit, listed in Chapter 17.
Export library to file	High	Insurance against the data-loss risk in Chapter 16. Users can currently lose everything with no warning.
Extract shared modules	High	lib/board.js, lib/tree.js, lib/engine.js. Removes the duplication documented in Chapter 20.
XSS audit	Medium	Verify every PGN-to-DOM path usestextContent.
Self-host chess.js	Medium	Removes the CDN supply-chain dependency.
Engine strength levels	Medium	UCI_LimitStrength and UCI_Elo, wired to the existing bot selector.

Version 2.0 — Mobile, and a stronger engine

The headline release. Two large pieces of work, both of which change what the product is capable of.

Item	Notes
Responsive rebuild	Retire the mobile gate. Fluid board sizing, collapsible sidebar, bottom-sheet analysis panel, touch-native piece movement. This is the largest single item on the roadmap and roughly doubles the addressable audience.
Multi-threaded Stockfish	Serve C00P and C0EP headers via a Cloudflare <code>_headers</code> file to unlock <code>SharedArrayBuffer</code> , then enable the <code>Threads</code> UCI option. Several times the strength at the same wall-clock time.
Full accessibility pass	ARIA roles on the board, a live region for move announcements, focus trapping in modals, <code>prefers-reduced-motion</code> . Done <i>with</i> the responsive work, not after it — they are the same work (Chapter 17).
Board and piece themes	The <code>getPieceImage()</code> seam already exists.
Game review mode	Walk a finished game move by move, classifying each against the engine's preference — blunder, mistake, inaccuracy, best. The single most requested feature in comparable tools.
Evaluation graph	A line chart of the evaluation across the whole game.
Serialise variations on save	Fixes the limitation documented in Chapter 12.
Service worker	Full offline support after first visit.

Version 3.0 — Accounts and the cloud

IMPORTANT

This is where the founding constraint is deliberately broken. Every feature in this release requires a backend, and adding one means giving up the properties that make Cheese distinctive: instant load, zero cost, and analysis that provably never leaves the device. The trade should only be made if cross-device sync is demonstrably what users actually want — and the local-only mode must remain fully functional for those who do not.

Item	Notes
User accounts	Authentication and identity.
Cloud saves	The actual motivation: study on a laptop, review on a phone.
Opening statistics	Real master-game frequency and scoring data per position, which requires a database far larger than can be shipped as static JSON.
Cloud analysis	Deeper engine searches run server-side for users on weak hardware.
Endgame trainer	Tablebase-backed endgame drilling.
Opening trainer	Spaced-repetition drilling of a personal repertoire.

Version 4.0 — Platform

Item	Notes
Online multiplayer	Real-time play against other users. A substantial undertaking: matchmaking, a game server, anti-cheat.
AI coach	Natural-language explanation of <i>why</i> a move is a mistake, rather than a number. The most genuinely novel item on this list.
Lichess / Chess.com import	Pull a user's game history directly.
Engine comparison	Run two engines on the same position side by side.
Desktop application	An Electron or Tauri wrapper for native performance and a full-strength local engine.
Plugin system	Third-party extensions.

Sequencing principle

The ordering above is not arbitrary. Debt is paid before features are added (1.3 before 2.0), reach is expanded before capability is deepened (mobile in 2.0 before multiplayer in 4.0), and the architectural constraint is held for as long as it can be held usefully. A roadmap that adds features on top of unpaid debt produces a project that becomes progressively harder to change — which is how hobby projects die.

CHAPTER 23

Glossary

Chess terminology

Term	Definition
Centipawn (cp)	One hundredth of a pawn; the unit engines use for evaluation. +100 means White is a pawn up in compensation-adjusted terms.
ECO	Encyclopaedia of Chess Openings. A classification system dividing all openings into five volumes (A-E), each with a hundred numbered codes.
En passant	A capture in which a pawn takes an enemy pawn that has just advanced two squares, as though it had advanced only one.
Evaluation	The engine's numeric assessment of a position, positive for White.
FEN	Forsyth-Edwards Notation. A single-line string encoding a complete chess position: piece placement, side to move, castling rights, en passant target, and the move counters.
Fifty-move rule	A draw may be claimed if fifty moves pass with no capture and no pawn move.
Mainline	The primary sequence of moves in an analysis, as opposed to a variation.
Mate score	An engine evaluation expressing a forced mate in N moves rather than a material value. Displayed as M3.
PGN	Portable Game Notation. The standard text format for recording a chess game, consisting of tag headers and SAN movetext.
Ply	A single move by one side. A full move (White then Black) is two plies.
Principal variation (PV)	The sequence of moves the engine believes both sides will play; its 'best line'.
Promotion	A pawn reaching the last rank becomes a queen, rook, bishop, or knight.
SAN	Standard Algebraic Notation. The human-readable move format: Nf3, exd5, 0-0.
Stalemate	The side to move has no legal move but is not in check. A draw.
Threefold repetition	The same position occurring three times permits a draw claim.
Variation	An alternative line branching from a position in the mainline.

Technical terminology

Term	Definition
Debounce	Delaying an action until input has stopped for a given interval, so that a burst of events triggers the action once rather than many times.

Term	Definition
Glassmorphism	A visual style using translucent, blurred panels that appear to float above the background, as though frosted glass.
IntersectionObserver	A browser API that reports when an element enters or leaves the viewport, evaluated off the main thread. Used here for infinite scroll and reveal animations.
localStorage	Browser key-value storage, scoped to an origin, persisting across sessions. Strings only, roughly 5–10 MB, synchronous.
SharedArrayBuffer	A memory buffer shareable across threads, required for multi-threaded WebAssembly. Gated behind cross-origin isolation headers.
UCI	Universal Chess Interface. The plain-text protocol chess engines speak: position, go, info, bestmove.
WebAssembly (WASM)	A binary instruction format that runs at near-native speed in the browser. It is what makes a full-strength local chess engine possible.
Web Worker	A script running on a background thread, communicating with the main thread only by message passing. It is why the engine's search never freezes the interface.

CHAPTER 24

Appendix

A. Technology summary

Component	Choice	Version
Markup	HTML5	—
Styling	CSS3, hand-written (no framework)	—
Application logic	JavaScript (vanilla, no framework, no build step)	ES2020+
Chess rules	chess.js	0.10.3
Engine	Stockfish Lite (WebAssembly)	18
Opening data	Static ECO JSON (volumes A-E)	—
Game data	PGN files, parsed in-browser	—
Persistence	localStorage	—
Hosting	Cloudflare Pages	—
DNS / CDN	Cloudflare (proxied CNAME)	—
Version control	Git / GitHub	—

B. Storage keys

Key	Type	Purpose
cheeseSavedAnalyses	JSON array	The user's persistent analysis library.
cheeseImportPGN	String	One-shot PGN hand-off, Database → Analysis.
selectedOpening	String	One-shot opening hand-off, Explorer → Analysis.

C. UCI commands used

Command	Direction	Purpose
uci	→ engine	Announce the protocol and request the engine's options.
isready	→ engine	Block until the engine has finished initialising.
stop	→ engine	Abort the current search. Sent before every new position.
position fen <FEN>	→ engine	Set the position to analyse.
go depth 18	→ engine	Search to a fixed depth (Analysis).
go movetime 700	→ engine	Search for a fixed time (Training).

Command	Direction	Purpose
info ... depth ... score cp ...	← engine	Streaming search progress and evaluation.
info ... score mate N	← engine	A forced mate has been found.
bestmove <move>	← engine	The search is complete. Used by Training to make the bot's move.

D. Resources and references

Resource	Relevance
Stockfish — stockfishchess.org	The engine. Also the source of the GPL obligations described below.
chess.js — github.com/jhlywa/chess.js	Move generation, validation, FEN and PGN handling.
UCI protocol specification	The authoritative description of the engine protocol, including the score sign convention that caused Bug 1 in Chapter 21.
PGN specification (Standard, Portable Game Notation)	The format read by the Database and the import feature.
ECO (Encyclopaedia of Chess Openings)	The opening classification the Explorer is built on.
MDN — Web Workers, IntersectionObserver, localStorage	Reference for the three browser APIs the architecture depends on.
Cloudflare Pages documentation	Deployment, custom domains, and the _headers file needed for the 2.0 multi-threading work.

E. Licence

Cheese is released under the **GNU General Public License v3.0**.

This is not an arbitrary choice. Stockfish is itself GPL-licensed, and distributing a work that incorporates it requires the combined work to be released under the same terms. The obligations that follow are worth stating plainly, because they apply to anyone who forks this project:

- The source must remain available under the GPL.
- Distributed derivative works must also be GPL-licensed.
- Copyright and licence notices must be preserved.
- Charging money is permitted — the GPL is not a non-commercial licence — but the source must still be available to those you distribute to. You may sell it; you may not close it.

NOTE

This matters for any future monetisation. Advertising and donations are entirely compatible with the GPL. A closed-source premium tier built on this codebase is not.

F. Credits and acknowledgements

Author	Vihaan Productions™
Engine	The Stockfish developers and contributors, whose work makes a free, full-strength local engine possible at all.
Rules library	Jeff Hlywa and the chess.js contributors.
Opening data	The ECO classification, as compiled by the chess community.
Hosting	Cloudflare Pages.

Cheese — a modern web-based chess analysis platform powered by Stockfish.
Version 1.2 — usecheese.xyz — GPL-3.0 — Vihaan Productions™